MEG-004 — Hexagonal Architecture

External publication draft

MEG-004 — Hexagonal Architecture

> *The Domain should know nothing about the outside world. The outside world should adapt itself to the Domain.*

Purpose

Mosaic is designed to evolve for many years.

During that time:

- databases will change
- APIs will change
- protocols will change
- storage engines will change
- extensions will change
- user interfaces will change

The business, however, should remain stable.

Hexagonal Architecture provides the structural rules that allow the Domain Model to remain independent of every external technology.

Unlike traditional layered architectures, Hexagonal Architecture does not organise software around technical layers.

Instead, it organises software around **dependencies**.

Everything outside the Domain adapts to it.

The Domain adapts to nothing.

Relationship to MEG

```
MEG-001

↓

Engineering Standards

↓

MEG-002

↓

Reactive Runtime

↓

MEG-003

↓

Domain Model

↓

MEG-004

↓

Dependency Boundaries

↓

Infrastructure
```

MEG-001 defines **how software is written.**

MEG-002 defines **how software executes.**

MEG-003 defines **what the business is.**

MEG-004 defines **how the business is protected from technology.**

Scope

This specification defines:

- Hexagonal philosophy
- Ports
- Adapters
- Driving adapters
- Driven adapters
- Dependency inversion
- Composition root
- Dependency direction
- Application services

- Infrastructure boundaries
- External systems
- Runtime integration
- Testing boundaries

This specification intentionally does **not** define:

- Business models
- Domain behaviour
- Runtime semantics
- Storage implementation
- Deployment architecture

Those concerns are defined by other MEG specifications.

Core Question

MEG-004 exists to answer one question.

> ***How should the Mosaic Domain Model remain completely independent of infrastructure while still interacting with the outside world?***

Architectural Statement

Within Mosaic:

> ***The Domain owns the contracts. Infrastructure provides the implementations.***

Everything external to the Domain is replaceable.

The Domain is not.

The Domain should never depend upon:

- PostgreSQL
- DuckDB
- HTTP
- REST
- GraphQL
- WebSockets
- Docker
- Jellyfin
- TMDB
- Stremio

- Trakt
- Message Brokers

Instead, those technologies adapt themselves to the Domain through Ports and Adapters.

Hexagonal Architecture (also known as Ports and Adapters) exists specifically to isolate business logic from external technologies through dependency inversion. [oai_citation:0‡AWS Documentation](https://docs.aws.amazon.com/prescriptive-guidance/latest/cloud-design-patterns/hexagonal-architecture.html?utm_source=chatgpt.com)

Architectural Hierarchy

The Mosaic Architecture intentionally separates concerns into distinct conceptual layers.

```
Business Domain

↓

Application

↓

Ports

↓

Adapters

↓

External Systems
```

Notice:

Dependencies flow **upwards**.

Knowledge flows **inwards**.

Infrastructure never leaks into the Domain.

Expected Outcome

After reading MEG-004 contributors should understand:

- why Hexagonal Architecture exists
- what Ports represent
- what Adapters implement
- how dependencies flow
- how infrastructure integrates with the Domain
- how runtime and domain remain independent
- how new technologies are introduced without modifying business logic

without discussing any specific database, transport protocol or framework.

Repository Structure

```
engineering/

■■■ meg/

    ■■■ MEG-004 Hexagonal Architecture/

        README.md

        00-document-control.md

        01-hexagonal-philosophy.md

        02-ports.md

        03-driving-ports.md

        04-driven-ports.md

        05-adapters.md

        06-driving-adapters.md

        07-driven-adapters.md

        08-dependency-direction.md

        09-composition-root.md

        10-application-services.md

        11-runtime-boundary.md

        12-testing-the-hexagon.md

        13-modelling-guidelines.md

        14-adrs.md

        15-contributor-guidance.md

        glossary.md

        references.md
```

Dependencies

Required reading:

- MEG-001 Go Engineering Standards
- MEG-002 Reactive Runtime
- MEG-003 Domain-Driven Design

Future companion specifications:

- MEG-005 Extension Platform
- MEG-006 Runtime Architecture
- MEG-007 Storage Architecture

Design Goals

The Hexagonal Architecture is intended to produce software that is:

- Technology independent
- Testable
- Evolvable
- Replaceable
- Decoupled
- Domain centric
- Explicit
- Maintainable

The architecture should ensure that changing infrastructure never requires changing the Domain Model.

Review Status

Status

Draft

Owner

Lead Software Architect

Next File

`00-document-control.md`

Document Control

Document Information

Field	Value
Document ID	MEG-004
Title	Hexagonal Architecture
File	00-document-control.md
Status	Draft
Version	0.1
Owner	Lead Software Architect
Classification	Internal Architecture Specification

Purpose

This document establishes the governance, authority and lifecycle of the Mosaic Hexagonal Architecture specification.

MEG-004 defines the architectural rules governing how the Domain Model interacts with infrastructure.

Unlike implementation documentation, this specification defines **dependency boundaries**, not implementation details.

Its primary purpose is to ensure that the Domain remains independent of technology for the lifetime of the Mosaic platform.

Authority

MEG-004 is the authoritative specification governing architectural boundaries throughout the Mosaic ecosystem.

This specification applies to:

- Mosaic Core
- First-party Extensions
- Third-party Extensions
- Runtime Capabilities
- SDKs
- Infrastructure Components

Every capability developed within the Mosaic platform SHOULD comply with the dependency rules established by this specification.

Relationship to Other Specifications

MEG specifications intentionally build upon one another.

```
MDL
↓
MDS
↓
MEG-001
↓
MEG-002
↓
MEG-003
↓
MEG-004
```

Specifically:

- **MDL** defines product philosophy.
- **MDS** defines presentation.
- **MEG-001** defines engineering standards.
- **MEG-002** defines runtime behaviour.
- **MEG-003** defines business modelling.
- **MEG-004** defines dependency boundaries.

Future specifications build upon the architectural separation established here.

Normative Language

Unless explicitly stated otherwise, the following keywords are interpreted according to RFC 2119.

Keyword	Meaning
MUST	Mandatory requirement.
MUST NOT	Prohibited behaviour.
SHOULD	Strong recommendation. Deviation requires architectural justification.
SHOULD NOT	Discouraged except where clearly justified.
MAY	Optional behaviour based upon engineering judgement.

Examples and diagrams are informative unless explicitly identified as normative.

Architectural Principles

The Mosaic Hexagonal Architecture is built upon several foundational principles.

- The Domain owns business behaviour.
- Dependencies flow towards the Domain.
- Infrastructure adapts to the Domain.
- Ports define contracts.
- Adapters implement contracts.
- Technology remains replaceable.
- Business logic remains isolated.
- The Domain never imports infrastructure.

Every subsequent chapter expands one or more of these principles.

Document Lifecycle

MEG specifications evolve alongside the platform.

Each document progresses through the following lifecycle.

```
Draft

↓

Review

↓

Accepted

↓

Implemented

↓

Maintained

↓

Superseded (optional)
```

Accepted specifications become part of the canonical Mosaic architecture.

Historical versions SHOULD remain available for future reference.

Architectural Evolution

Hexagonal Architecture is intentionally stable.

Changes affecting:

- dependency direction

- port definitions
- adapter responsibilities
- application services
- runtime integration
- infrastructure boundaries

SHOULD be accompanied by an Architectural Decision Record (ADR).

Architectural consistency should remain more important than implementation convenience.

Compliance

All repositories implementing Mosaic capabilities SHOULD comply with MEG-004.

Where deviation becomes necessary, the repository SHOULD document:

- the reason
- affected boundaries
- architectural impact
- migration strategy

Temporary deviations should eventually be removed.

Permanent deviations should generally result in updates to this specification.

Design Philosophy

MEG-004 intentionally favours:

- explicit dependencies
- replaceable infrastructure
- domain independence
- clear ownership
- technology isolation
- long-term maintainability

The architecture should continue functioning even if:

- databases change
- frameworks change
- transports change
- infrastructure changes

Only the adapters should require modification.

The Domain should remain unchanged.

Hexagonal Architecture exists precisely to achieve this separation between business logic and infrastructure by ensuring dependencies point inward towards the application core.
([alistair.cockburn.us](https://alistair.cockburn.us/hexagonal-architecture))

Scope of Authority

MEG-004 governs architectural boundaries.

It does **not** define:

- business behaviour
- runtime execution
- storage technologies
- deployment topology
- user interface design

Those concerns belong to other MEG specifications.

Keeping these concerns separate allows each architectural layer to evolve independently.

Review Status

Status

Draft

Owner

Lead Software Architect

Previous File

`README.md`

Next File

`01-hexagonal-philosophy.md`

Hexagonal Philosophy

> *Technology is temporary. The business is not.*

Purpose

Software inevitably changes.

Databases are replaced.

Protocols evolve.

Frameworks become obsolete.

Programming languages change.

The business, however, generally changes far more slowly.

Hexagonal Architecture exists to ensure that technological change does not force unnecessary business change.

Within Mosaic, the Domain Model should remain the most stable part of the entire platform.

Everything else exists to support it.

Philosophy

Within Mosaic:

> ***The Domain should know nothing about how it is used or how it is persisted.***

The Domain should not know:

- HTTP
- PostgreSQL
- DuckDB
- Blob Storage
- Event Bus
- Docker
- TMDB
- Jellyfin
- Stremio

Instead, those technologies adapt themselves to the Domain.

Never the reverse.

The Problem

Traditional layered architectures frequently evolve into this.

```
HTTP

↓

Service

↓

Repository

↓

Database

↓

Business Logic
```

Over time the business becomes increasingly coupled to infrastructure.

Examples include:

- SQL exceptions inside business logic
- HTTP concepts inside entities
- framework annotations inside the domain
- persistence models becoming business models

Eventually:

Changing infrastructure requires changing business behaviour.

This is precisely what Hexagonal Architecture seeks to prevent.

The Core Principle

Everything revolves around one idea.

```
Dependencies Always Point Inward
```

Not outward.

The Domain owns:

- business rules
- business language
- business behaviour

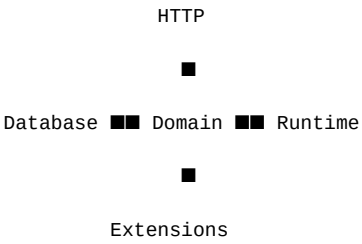
Infrastructure depends upon those concepts.

The Domain depends upon nothing external.

This inversion of dependency direction is the defining characteristic of Ports and Adapters architecture. ([alistair.cockburn.us](https://alistair.cockburn.us/hexagonal-architecture))

The Hexagon

The architecture is traditionally represented as a hexagon.



The shape itself is symbolic.

It communicates one important idea.

There is no "top" or "bottom."

Every external system is simply another adapter.

HTTP is not special.

Neither is the database.

Inside The Hexagon

The inside contains only business concepts.

Examples include:

- Entities
- Value Objects
- Aggregates
- Domain Services
- Domain Events

The Domain should be understandable without mentioning any technology.

If infrastructure terminology appears inside the Domain:

The architectural boundary has already been breached.

Outside The Hexagon

Everything external becomes infrastructure.

Examples include:

- HTTP
- REST
- GraphQL
- PostgreSQL

- DuckDB
- Redis
- Blob Storage
- Event Bus
- Docker
- CLI
- Mobile Applications

These technologies change frequently.

The Domain should remain unaffected.

The Domain Is The Product

One of the central principles of Mosaic is:

```
Business
```

↓

```
Technology
```

Not:

```
Technology
```

↓

```
Business
```

The platform exists because of:

- Libraries
- Playback
- Metadata
- Users

Not because of PostgreSQL.

Infrastructure should therefore remain replaceable.

Replaceability

Suppose PostgreSQL becomes unsuitable.

Without Hexagonal Architecture.

```
Database Changes
```

↓

```
Repositories
```

↓

```
Services
↓
Entities
↓
Everything Changes
```

With Hexagonal Architecture.

```
Database Changes
↓
Postgres Adapter
↓
New Adapter
↓
Domain Unchanged
```

Only infrastructure changes.

The business remains stable.

Runtime Independence

Likewise:

The Domain should not know the Reactive Runtime exists.

Poor.

```
Playback
↓
Publish Event
↓
Event Bus
```

Preferred.

```
Playback
↓
Raise Domain Event
```

Later.

```
Runtime Adapter
↓
Runtime Event
↓
Event Bus
```

The Domain records business facts.

The Runtime communicates them.

The separation established in MEG-002 remains intact.

Technology Independence

The Domain should remain capable of operating without:

- databases
- networks
- filesystems
- messaging
- containers

If a Domain Model cannot be unit tested without PostgreSQL:

The architecture has failed.

The Cost Of Coupling

Infrastructure changes frequently.

Examples include:

```
TMDB API

↓

New Version

Storage Engine

↓

Migration

Docker

↓

New Runtime
```

These changes should affect adapters.

Not business behaviour.

Every dependency from the Domain to infrastructure increases future maintenance cost.

Business Before Infrastructure

When designing new functionality ask:

> ***What is the business behaviour?***

Only afterwards ask:

> ***How will the infrastructure support it?***

The second question should never influence the answer to the first.

Every Technology Is Equal

One subtle but important property of the Hexagon:

```
HTTP

=

CLI

=

Worker

=

Scheduler

=

Event Bus

=

Tests
```

All are simply mechanisms for interacting with the Domain.

No technology occupies a privileged position.

The Domain remains the centre.

Infrastructure Is Temporary

History demonstrates that infrastructure changes.

Examples.

```
REST

↓

GraphQL

Docker

↓

Containerd

PostgreSQL

↓

Something Else
```

Business concepts such as:

```
Library
```

```
Playback
```

```
Collection
```

rarely disappear.

Architecture should therefore optimise for the longer-lived concepts.

Ports Before Adapters

One of the defining ideas of Hexagonal Architecture is:

The Domain defines contracts.

Infrastructure satisfies them.

Never:

```
Database
```

```
↓
```

```
Business
```

Instead.

```
Business
```

```
↓
```

```
Port
```

```
↓
```

```
Adapter
```

```
↓
```

```
Database
```

This inversion keeps the Domain in control.

Simplicity

Hexagonal Architecture should simplify software.

It should not introduce unnecessary abstraction.

If a Port exists:

It should represent genuine business interaction.

Not hypothetical future flexibility.

MEG-001's principles still apply.

Concrete solutions remain preferable until abstraction becomes necessary.

Mosaic Principles

Within Mosaic:

- The Domain owns the architecture.
- Infrastructure adapts to the Domain.
- Dependencies always point inward.
- Technology remains replaceable.
- Runtime remains infrastructure.
- Extensions remain infrastructure.
- Storage remains infrastructure.
- Transport remains infrastructure.
- Business behaviour remains independent.

These principles define the architectural identity of the platform.

Relationship to MEG

MEG-003 established:

> ***What the business is.***

MEG-004 begins answering:

> ***How do we protect it?***

The remaining chapters explain the mechanisms through which Hexagonal Architecture preserves Domain independence.

The first of those mechanisms is the **Port**.

Summary

Hexagonal Architecture is often misunderstood as a folder structure.

It is not.

It is a dependency philosophy.

Within Mosaic it exists for one reason:

> ***To ensure the Domain remains the most stable, valuable and protected part of the entire platform.***

Everything else is expected to change.

The Domain should not.

Review Status

Status

Draft

Owner

Lead Software Architect

Previous File

`00-document-control.md`

Next File

`02-ports.md`

Ports

> *A Port defines what the Domain needs. It never defines how that need is fulfilled.*

Purpose

The Domain must interact with the outside world.

Examples include:

- loading Aggregates
- publishing Domain Events
- reading metadata
- writing blob storage
- retrieving configuration
- querying external providers

However, the Domain should remain completely unaware of:

- PostgreSQL
- DuckDB
- TMDB
- Jellyfin
- Docker
- HTTP
- Filesystems

Ports solve this problem.

They define the contracts through which the Domain communicates with external systems without depending upon those systems.

Philosophy

Within Mosaic:

> ***The Domain owns every contract it depends upon.***

A Port represents a business capability.

It does **not** represent a technology.

Infrastructure implements Ports.

The Domain defines them.

This inversion of ownership is the defining characteristic of Hexagonal Architecture.

What Is A Port?

A Port is an interface representing a business capability required by the Domain.

Examples include:

```
MediaRepository

MetadataProvider

ArtworkStore

Clock
```

Each Port answers one question.

> ***What does the Domain require?***

Not:

> ***How is it implemented?***

Why Ports Exist

Without Ports:

```
Playback

↓

PostgreSQL

↓

SQL

↓

Database
```

The Domain now understands infrastructure.

Instead:

```
Playback

↓

PlaybackRepository

↓

PostgreSQL Adapter
```

The Domain understands only:

```
PlaybackRepository
```

Everything else becomes replaceable.

The Domain Owns The Port

One of the most important principles of Hexagonal Architecture is:

```
Domain

↓

Defines Interface

↓

Infrastructure Implements
```

Not:

```
Infrastructure

↓

Defines Interface

↓

Domain Uses
```

Ownership always belongs to the Domain.

The business decides what it needs.

Infrastructure satisfies those needs.

Ports Describe Behaviour

Ports should describe business behaviour.

Good.

```
[go]
type PlaybackRepository interface {

    FindByID(...)

    Save(...)
}
```

Poor.

```
[go]
type PostgreSQLRepository interface {

    ExecuteSQL(...)
}
```

The first describes business intent.

The second describes implementation.

Ports should never leak technology.

Business Language

Port names should reinforce the ubiquitous language.

Good.

```
CollectionRepository

MetadataProvider
```

```
RecommendationEngine
```

Poor.

```
DatabaseAccess

StorageLayer

PersistenceManager
```

Business concepts should always dominate technical vocabulary.

Ports Are Stable

Infrastructure changes frequently.

Ports should not.

Changing a Port affects:

- the Domain
- every Adapter
- every test
- every implementation

Ports therefore form part of the long-term architectural contract.

They should evolve deliberately.

Ports Are Small

Ports SHOULD remain focused.

Good.

```
[go]
type ArtworkProvider interface {

    Artwork(...)
}
```

Poor.

```
[go]
type MediaPlatform interface {

    Import()

    Search()

    Playback()

    Metadata()

    Recommendations()

    Authentication()
}
```

Large Ports increase coupling.

Small Ports improve flexibility.

This aligns closely with Go's preference for small interfaces representing behaviour rather than broad capability sets. ([go.dev](https://go.dev/doc/effective_go?utm_source=chatgpt.com))

Business First

A useful question when designing a Port is:

> ***What would the business ask for?***

Not:

> ***What can PostgreSQL provide?***

The Domain should remain entirely unaware of implementation constraints.

Technology Neutral

Ports should never reference:

- SQL
- REST
- HTTP
- Kafka
- NATS
- Docker
- Redis

Poor.

`SQLRepository`

Good.

`MediaRepository`

Technology belongs behind the Port.

Ports Are Intent

Ports communicate intent.

Example.

`MetadataProvider`

communicates:

> *Retrieve metadata.*

It says nothing about:

- TMDB
- AniList
- Local Cache
- Filesystem

Those become Adapter concerns.

One Responsibility

Each Port SHOULD describe one responsibility.

Examples.

`MetadataProvider`

`ArtworkStore`

`Clock`

Avoid combining unrelated concepts.

Poor.

`MediaInfrastructure`

One Port should answer one business question.

Domain Dependencies

Everything the Domain depends upon should enter through Ports.

Examples include:

- repositories
- providers
- storage
- time
- identity generation

This dramatically improves:

- testing
- replacement
- evolution

The Domain remains isolated.

Input vs Output

Not all Ports are identical.

Some receive requests.

Others perform work on behalf of the Domain.

Hexagonal Architecture traditionally distinguishes these as:

- Driving Ports
- Driven Ports

The next two chapters explore this distinction.

Ports Are Not Adapters

A common mistake is confusing:

`Port`

with

`Adapter`

Ports define contracts.

Adapters implement contracts.

The Port belongs to the Domain.

The Adapter belongs to Infrastructure.

The two should never be combined.

Testing

Ports make testing straightforward.

Example.

`PlaybackRepository`

↓

`Fake Repository`

The Domain remains unaware that no real database exists.

Tests become:

- deterministic
- fast
- infrastructure independent

This is one of the primary practical benefits of Hexagonal Architecture.

Port Evolution

Ports should evolve slowly.

Whenever a Port changes ask:

- Is the Domain changing?
- Or merely the infrastructure?

If only infrastructure changed:

The Port probably should not.

Ports should remain stable as technologies evolve.

Mosaic Examples

Examples of Ports within Mosaic include:

`LibraryRepository`

`PlaybackRepository`

`MetadataProvider`

`ArtworkStore`

`BlobStore`

`IdentityGenerator`

`Clock`

Every Port expresses a business dependency.

None express technology.

Anti-Patterns

The following practices are prohibited.

Technology Ports

`PostgresRepository`

`RedisProvider`

Generic Ports

`Storage`

without business meaning.

Large Ports

Interfaces representing unrelated capabilities.

Infrastructure Dependencies

Ports importing:

- SQL
- HTTP
- Runtime
- Logging

Shared Ownership

Infrastructure defining contracts consumed by the Domain.

The Domain owns the contract.

Always.

Mosaic Guidelines

Within Mosaic:

- Ports MUST belong to the Domain.
- Ports MUST describe business behaviour.
- Ports MUST remain technology independent.
- Ports SHOULD remain small.
- Ports SHOULD reinforce the ubiquitous language.
- Infrastructure MUST implement Ports.
- Ports SHOULD evolve slowly.
- Business requirements MUST drive Port design.

Relationship to MEG

Hexagonal Architecture revolves around one simple idea.

```
Domain
↓
Port
↓
Adapter
↓
Technology
```

Ports define the contracts.

Adapters satisfy them.

The next chapter introduces the first category of Ports:

Driving Ports, which define how the outside world requests business behaviour from the Domain.

Summary

Ports are one of the most important concepts within Hexagonal Architecture.

They invert the traditional ownership of dependencies.

Instead of infrastructure telling the Domain how to behave:

The Domain tells infrastructure what it requires.

That inversion protects the business from technology and ensures the Domain remains the most stable part of the Mosaic platform.

Review Status

Status

Draft

Owner

Lead Software Architect

Previous File

`01-hexagonal-philosophy.md`

Next File

`03-driving-ports.md`

Driving Ports

> *Driving Ports describe what the outside world may ask the Domain to do.*

Purpose

Every business capability exposes behaviour.

Examples include:

- importing media
- starting playback
- creating collections
- updating metadata
- authenticating users

External systems require a mechanism through which they can request that behaviour.

Within Hexagonal Architecture, these contracts are known as **Driving Ports**.

Driving Ports define the public capabilities of the Domain.

They do **not** define how those capabilities are invoked.

Philosophy

Within Mosaic:

> ***Driving Ports express business use cases, not transport protocols.***

A Driving Port answers one question.

> ***What business behaviour is available?***

It does not answer:

> ***How does someone invoke it?***

Transport remains an infrastructure concern.

What Is A Driving Port?

A Driving Port is an interface through which an external actor requests business behaviour.

Examples include:

- HTTP
- CLI
- Scheduler
- Event Subscriber

- Extension
- Test

All interact with the Domain through the same Driving Port.

This separation allows the Domain to remain unaware of how requests arrive.

Why Driving Ports Exist

Without Driving Ports:

```
HTTP

↓

Playback

↓

Business
```

Later.

```
CLI

↓

Different Business Logic
```

Eventually.

```
Worker

↓

Another Implementation
```

Business behaviour becomes duplicated.

Instead.

```
HTTP

↓

Playback Port

↓

Domain

CLI

↓

Playback Port

↓

Domain

Worker

↓

Playback Port
```

↓

`Domain`

Every entry point invokes exactly the same business behaviour.

The Domain Defines Behaviour

Driving Ports belong to the Application layer immediately surrounding the Domain.

They describe:

- use cases
- commands
- business operations

They do **not** describe:

- HTTP endpoints
- REST resources
- WebSocket messages
- CLI commands

Technology adapts itself to the Port.

Never the reverse.

One Port Per Use Case

Driving Ports should model business capabilities.

Good.

`PlaybackService`

`LibraryImporter`

`CollectionManager`

Poor.

`HTTPPlaybackController`

`RESTLibraryAPI`

The business should remain unaware of transport.

Use Cases

Every operation exposed by a Driving Port should correspond to a business use case.

Examples.

`ImportMedia()`

`ResumePlayback()`

```
CreateCollection()

GenerateRecommendations()
```

Notice:

These operations describe business behaviour.

Not infrastructure.

Business Language

Driving Ports should reinforce the ubiquitous language.

Good.

```
[go]
ResumePlayback(...)
```

Poor.

```
[go]
ExecutePlaybackHandler(...)
```

The Port should read like a conversation with the business.

Request Models

Driving Ports may define request objects.

Example.

```
[go]
type ImportMediaRequest struct {

    LibraryID LibraryID

    Source Source
}
```

These request models represent business concepts.

They should not mirror:

- HTTP payloads
- JSON documents
- database schemas

Transport models should be translated before reaching the Port.

Response Models

Likewise:

Driving Ports may return business responses.

Example.

```
[go]
```

```
type ImportMediaResult struct {

    MediaID MediaID
}
```

Avoid exposing:

- HTTP status codes
- JSON responses
- protobuf messages

The Domain should return business concepts.

Transport decides how to present them.

Commands

Driving Ports frequently execute commands.

Example.

```
Import Media

↓

ImportMedia()
```

A Driving Port represents the intention to perform business work.

Successful execution may later produce Domain Events.

One Behaviour

Driving Port operations should remain cohesive.

Poor.

```
[go]
ProcessEverything()
```

Better.

```
[go]
ImportMedia()

[go]
CreateCollection()

[go]
ArchiveMedia()
```

Every operation should communicate one business intention.

No Infrastructure

Driving Ports MUST remain infrastructure agnostic.

Poor.

```
[go]
ImportMedia(http.Request)
```

Better.

```
[go]
ImportMedia(request ImportMediaRequest)
```

HTTP belongs to the Adapter.

The Port belongs to the Domain.

Multiple Adapters

One Driving Port may have many Adapters.

Example.

```
Playback Port

■■■ HTTP Adapter

■■■ CLI Adapter

■■■ Scheduler Adapter

■■■ Extension Adapter

■■■ Test Adapter
```

The Domain remains unchanged.

Only the Adapters differ.

This is one of the key advantages of Hexagonal Architecture.

Validation

Driving Ports should receive already valid transport models.

Examples.

```
HTTP

↓

Parse JSON

↓

Validate JSON

↓

Driving Port
```

Business validation still occurs inside the Domain.

Transport validation remains outside.

The two solve different problems.

Transactions

Driving Ports define business operations.

They do not own:

- database transactions
- retries
- event publication
- infrastructure coordination

Those responsibilities belong elsewhere within the architecture.

The Port simply exposes business behaviour.

Testing

Driving Ports make business testing straightforward.

Example.

```
Test

↓

Driving Port

↓

Domain
```

No HTTP server.

No REST.

No database.

The business can be exercised directly.

This dramatically simplifies testing.

Event Subscribers

Within Mosaic's Reactive Runtime, event subscribers frequently become Driving Adapters.

Example.

```
PlaybackCompleted Event

↓

Subscriber Adapter

↓

Driving Port

↓
```

`Recommendations Domain`

Notice:

The subscriber remains infrastructure.

The Domain still receives business requests through the Driving Port.

This cleanly integrates MEG-002 with Hexagonal Architecture.

Examples Within Mosaic

Examples of Driving Ports include:

`PlaybackService`

`LibraryImporter`

`CollectionService`

`MetadataManager`

`RecommendationEngine`

Each exposes business behaviour.

None expose technology.

Anti-Patterns

The following practices are prohibited.

HTTP In Ports

```
[go]
ServeHTTP(...)
```

Database Models

Ports accepting SQL entities.

JSON Objects

Ports exposing transport models directly.

Framework Types

Ports importing:

- gin
- echo
- grpc
- protobuf

Generic Methods

`Execute()`

`Handle()`

`Process()`

without clear business meaning.

Mosaic Guidelines

Within Mosaic:

- Driving Ports MUST expose business use cases.
- Driving Ports MUST remain transport independent.
- Driving Ports SHOULD reinforce the ubiquitous language.
- Every operation SHOULD describe one business behaviour.
- Request and response models SHOULD represent business concepts.
- Multiple adapters MAY implement the same Driving Port.
- Infrastructure MUST translate before invoking the Port.
- Driving Ports MUST remain stable as transport evolves.

Relationship to MEG

Ports define contracts.

Driving Ports define:

> ***How the outside world requests business behaviour.***

The next chapter introduces **Driven Ports**, which define the opposite direction:

> ***How the Domain requests capabilities from the outside world.***

Together they complete the communication boundary between the Domain and infrastructure.

Summary

Driving Ports represent the public face of the Domain.

They describe:

- business capabilities
- business language
- business intent

They remain completely independent of:

- HTTP

- CLI
- Runtime
- Extensions
- Transport

By ensuring every external interaction passes through the same business contract, Mosaic guarantees that changing how users interact with the platform never requires changing the Domain itself.

Review Status

Status

Draft

Owner

Lead Software Architect

Previous File

`02-ports.md`

Next File

`04-driven-ports.md`

Driven Ports

> *Driven Ports describe the capabilities the Domain requires from the outside world. They never describe how those capabilities are implemented.*

Purpose

Business behaviour frequently depends upon external capabilities.

Examples include:

- loading Aggregates
- persisting changes
- retrieving metadata
- storing artwork
- generating identifiers
- obtaining the current time

The Domain requires these capabilities.

It should not know how they are fulfilled.

Driven Ports define these requirements.

They represent the contracts through which the Domain requests services from infrastructure while remaining completely independent of implementation.

Philosophy

Within Mosaic:

> ***The Domain expresses needs. Infrastructure fulfils them.***

A Driven Port answers one question.

> ***What capability does the Domain require?***

It never answers:

> ***Which technology provides it?***

Implementation remains outside the Domain.

What Is A Driven Port?

A Driven Port is an interface representing a capability required by the Domain.

Examples include:

```
LibraryRepository
```

```
MetadataProvider
```

```
ArtworkStore

Clock

IdentityGenerator
```

Each describes behaviour.

None describe technology.

Why Driven Ports Exist

Without Driven Ports:

```
Playback

↓

PostgreSQL

↓

SQL

↓

Persistence
```

The Domain now understands infrastructure.

Instead.

```
Playback

↓

PlaybackRepository

↓

PostgreSQL Adapter
```

Only the Adapter knows PostgreSQL exists.

The Domain remains infrastructure independent.

Dependency Inversion

Driven Ports are the mechanism through which Dependency Inversion is achieved.

```
Domain

↓

Interface

↓

Infrastructure
```

Not:

```
Infrastructure

↓
```

```
Interface
```

↓

```
Domain
```

The Domain owns every dependency it requires.

Infrastructure adapts itself to satisfy those contracts.

The Domain Defines Requirements

A Driven Port expresses a business requirement.

Example.

```
PlaybackRepository
```

The Domain requires:

```
Load Playback
```

```
Save Playback
```

It does **not** require:

```
SQL
```

```
Transactions
```

```
Indexes
```

```
Connection Pools
```

Those concerns belong entirely to infrastructure.

Ports Describe Behaviour

Driven Ports should describe business behaviour.

Good.

```
[go]
type MetadataProvider interface {

    Metadata(...)
}
```

Poor.

```
[go]
type TMDBClient interface {

    GetMovie(...)
}
```

The first describes business intent.

The second exposes infrastructure.

Business Language

Driven Ports should reinforce the ubiquitous language.

Good.

```
ArtworkStore

IdentityGenerator

RecommendationRepository
```

Poor.

```
BlobClient

DatabaseAccess

RedisCache
```

Technology names should never appear within the Domain.

Multiple Implementations

One Driven Port may have many implementations.

Example.

```
MetadataProvider

■■■ TMDB Adapter

■■■ AniList Adapter

■■■ Local Cache Adapter

■■■ Test Adapter
```

The Domain remains unchanged.

Adapters become interchangeable.

This flexibility is one of the primary architectural benefits of Hexagonal Architecture.

Repositories

Repositories are among the most common Driven Ports.

Example.

```
Playback Domain

↓

PlaybackRepository

↓

PostgreSQL
```

The Domain depends upon:

```
PlaybackRepository
```

Only.

Persistence remains completely replaceable.

External Providers

External services should always appear behind Driven Ports.

Example.

```
Metadata Domain

↓

MetadataProvider

↓

TMDB
```

Later.

```
MetadataProvider

↓

AniList
```

The Domain does not change.

Only the Adapter changes.

Time

Time is infrastructure.

Poor.

```
[go]
time.Now()
```

inside the Domain.

Preferred.

```
Clock

↓

Current Time
```

The Domain requests:

```
Current Time
```

The infrastructure determines how that time is obtained.

This improves both testing and determinism.

Identity Generation

Identity generation should also occur through a Driven Port.

Example.

```
IdentityGenerator
```

↓

```
Generate LibraryID
```

The Domain requires:

```
Unique Identity
```

It does not require:

- UUID
- ULID
- Snowflake
- Database Sequence

Those are implementation choices.

Storage

Storage technologies should never appear within the Domain.

Poor.

```
Blob Storage
```

Preferred.

```
ArtworkStore
```

The business requires artwork storage.

It does not care whether artwork is stored in:

- Blob Storage
- Filesystem
- Cloud Storage

The Adapter owns that decision.

Ports Should Be Small

Driven Ports should remain narrowly focused.

Good.

```
ArtworkStore
```

Poor.

```
InfrastructureProvider
```

Each Port should represent one business capability.

Nothing more.

Ports Are Stable

Infrastructure changes frequently.

Driven Ports should remain relatively stable.

Changing a Port forces changes throughout:

- Domain
- Adapters
- Tests

Changing an Adapter affects only infrastructure.

Stable Ports protect the Domain from implementation churn.

Error Semantics

Driven Ports should communicate business failures.

Poor.

```
SQL Error
```

Preferred.

```
Media Not Found
```

The Adapter translates infrastructure failures.

The Domain receives business concepts.

This mirrors the repository guidance established in MEG-003.

Testing

Driven Ports make infrastructure easy to replace.

Example.

```
PlaybackRepository
```

↓

```
InMemoryRepository
```

```
Clock
```

↓

```
FixedClock
```

```
MetadataProvider
```

↓

```
FakeProvider
```

The Domain can now be tested without external dependencies.

Runtime Integration

The Reactive Runtime itself may satisfy Driven Ports.

Example.

```
Domain

↓

EventPublisher

↓

Runtime Adapter

↓

Event Bus
```

Notice:

The Domain depends only upon the Port.

The Runtime implements it.

MEG-002 and MEG-004 therefore complement one another naturally.

Anti-Corruption Layers

Driven Ports frequently terminate at Anti-Corruption Layers.

Example.

```
MetadataProvider

↓

TMDB Adapter

↓

TMDB API
```

Translation occurs entirely inside the Adapter.

The Domain never understands external terminology.

This preserves the purity of the Domain Model established in MEG-003.

Examples Within Mosaic

Examples of Driven Ports include:

```
LibraryRepository

PlaybackRepository
```

`MetadataProvider`

`ArtworkStore`

`Clock`

`IdentityGenerator`

`BlobStore`

Every Port expresses a business dependency.

None expose implementation.

Anti-Patterns

The following practices are prohibited.

Infrastructure Interfaces

`TMDBClient`

`PostgresRepository`

inside the Domain.

Framework Dependencies

Ports importing:

- SQL
- HTTP
- Redis
- Docker

Technology Language

Using implementation terminology rather than business terminology.

Generic Infrastructure Ports

`StorageProvider`

without business meaning.

Shared Infrastructure Contracts

Allowing infrastructure to define interfaces consumed by the Domain.

The Domain always owns its own contracts.

Mosaic Guidelines

Within Mosaic:

- Driven Ports MUST belong to the Domain.
- Driven Ports MUST express business requirements.
- Driven Ports MUST remain technology independent.
- Infrastructure MUST implement Driven Ports.
- Ports SHOULD remain focused and cohesive.
- External systems MUST be hidden behind Adapters.
- Error semantics SHOULD remain business oriented.
- Driven Ports SHOULD evolve more slowly than infrastructure.

Relationship to MEG

Driving Ports answer:

> ***How does the outside world invoke the Domain?***

Driven Ports answer:

> ***How does the Domain obtain capabilities from the outside world?***

Together they define every dependency crossing the boundary of the Hexagon.

The next chapter introduces **Adapters**, the infrastructure components that implement these contracts while keeping the Domain completely insulated from technology.

Summary

Driven Ports are the Domain's expression of dependency.

They communicate:

- what the business requires
- not how it is implemented

By ensuring every external dependency passes through a Domain-owned contract, Mosaic protects its most valuable asset:

The business model itself.

Infrastructure remains free to evolve.

The Domain remains free to ignore it.

Review Status

Status

Draft

Owner

Lead Software Architect

Previous File

`03-driving-ports.md`

Next File

`05-adapters.md`

Adapters

> *If Ports define what the Domain needs, Adapters define how the outside world fulfils those needs.*

Purpose

The Domain communicates exclusively through Ports.

However, Ports are only contracts.

Something must translate those contracts into real technologies such as:

- HTTP
- PostgreSQL
- DuckDB
- Blob Storage
- Event Bus
- Docker
- TMDB
- Jellyfin
- Stremio

Those translations are performed by **Adapters**.

Adapters isolate infrastructure from the Domain while allowing technologies to evolve independently.

Philosophy

Within Mosaic:

> ***Adapters translate. They do not decide.***

Adapters are translators.

They convert:

- transport into business requests
- business requests into infrastructure operations
- infrastructure responses into business concepts

They should never contain business rules.

Those belong to the Domain.

What Is An Adapter?

An Adapter implements a Port.

Conceptually.

```
Port
↓
Adapter
↓
Technology
```

Examples include:

```
PlaybackRepository
↓
PostgreSQL Adapter

MetadataProvider
↓
TMDB Adapter

ArtworkStore
↓
Blob Storage Adapter
```

The Adapter satisfies the Port.

The Domain remains unchanged.

Why Adapters Exist

Without Adapters:

```
Domain
↓
PostgreSQL
↓
SQL
↓
Storage
```

Business behaviour becomes coupled to infrastructure.

Instead:

```
Domain
↓
PlaybackRepository
↓
PostgreSQL Adapter
```

↓

```
Database
```

Only the Adapter understands SQL.

The Domain never does.

Translation Layer

An Adapter performs translation.

Example.

```
HTTP Request
```

↓

```
JSON
```

↓

```
Request DTO
```

↓

```
Business Request
```

↓

```
Driving Port
```

Or:

```
Business Request
```

↓

```
Repository Port
```

↓

```
SQL
```

↓

```
Database
```

Notice:

Translation occurs only inside the Adapter.

The Domain never sees infrastructure models.

Adapters Own Technology

Every technology belongs inside an Adapter.

Examples include:

- SQL
- HTTP

- GraphQL
- Redis
- Docker
- Kafka
- Blob Storage
- TMDB SDK
- Jellyfin API

If these concepts appear inside the Domain:

The architectural boundary has failed.

Business Objects Stay Inside

Adapters should convert infrastructure models into Domain concepts.

Poor.

```
SQL Row

↓

Domain
```

Better.

```
SQL Row

↓

Adapter

↓

Aggregate
```

Likewise.

```
HTTP JSON

↓

Adapter

↓

Business Request
```

The Domain should never parse JSON.

One Adapter, One Technology

Each Adapter SHOULD represent one integration.

Good.

```
TMDB Adapter
```

Poor.

```
MetadataAdapter

↓

TMDB

AniList

IMDb

Local Files
```

Multiple technologies should generally produce multiple Adapters implementing the same Port.

This keeps integrations isolated and independently replaceable.

Multiple Adapters

One Port may have many Adapters.

Example.

```
MetadataProvider

■■■ TMDB Adapter

■■■ AniList Adapter

■■■ Cached Adapter

■■■ Test Adapter
```

The Domain depends only upon:

```
MetadataProvider
```

Changing Adapters requires no Domain changes.

This is the core value proposition of Ports and Adapters. [oai_citation:0‡AWS Documentation](https://docs.aws.amazon.com/prescriptive-guidance/latest/cloud-design-patterns/hexagonal-architecture.html?utm_source=chatgpt.com)

Adapters Are Replaceable

Adapters should be considered disposable.

Suppose:

```
TMDB

↓

Deprecated
```

The replacement should require:

```
New Adapter
```

Not:

```
Domain Rewrite
```

Replaceability is one of the primary architectural goals.

Adapters Are Thin

Adapters SHOULD remain small.

Typical responsibilities include:

- translation
- mapping
- validation
- protocol conversion
- error translation

Adapters SHOULD NOT contain:

- business rules
- workflow decisions
- Aggregate logic
- invariants

If business behaviour appears inside an Adapter:

Move it into the Domain.

Error Translation

Adapters translate infrastructure failures into business concepts.

Poor.

```
SQL Error

↓

Domain
```

Preferred.

```
SQL Error

↓

Adapter

↓

Media Not Found
```

The Domain should never understand database exceptions.

Mapping

Adapters frequently perform mapping.

Examples include:

```
JSON

↓

Domain Request

Database Row

↓

Aggregate

TMDB Response

↓

Metadata Value Object
```

Mapping belongs entirely to infrastructure.

Not the Domain.

Runtime Integration

The Reactive Runtime integrates through Adapters.

Example.

```
Domain Event

↓

Runtime Adapter

↓

Runtime Event

↓

Event Bus
```

The Domain remains unaware that an Event Bus even exists.

The Adapter performs the translation.

This preserves the separation established in MEG-002.

External APIs

Every external API SHOULD terminate at an Adapter.

Example.

```
AniList

↓
```

```
AniList Adapter
↓
MetadataProvider
```

The Domain should never import:

- API clients
- SDKs
- REST models

The Adapter shields the Domain from external change.

Testing

Adapters are tested independently.

Typical tests verify:

- mapping correctness
- translation
- protocol handling
- infrastructure interaction

Domain tests should not require Adapter tests.

Responsibilities remain separate.

Composition Root

Adapters are assembled within the Composition Root.

Example.

```
main()
↓
Postgres Adapter
↓
PlaybackRepository
↓
Playback Domain
```

The Domain never constructs its own Adapters.

Construction belongs entirely outside the Hexagon.

Evolution

Adapters change frequently.

Examples include:

```
REST
↓
GraphQL

PostgreSQL
↓
CockroachDB

Blob Storage
↓
S3
```

The Adapter changes.

The Port remains.

The Domain remains.

This asymmetry is intentional.

Examples Within Mosaic

Examples of Adapters include:

```
HTTP Playback Adapter

CLI Import Adapter

PostgreSQL Playback Repository

DuckDB Analytics Repository

TMDB Metadata Adapter

Jellyfin Compatibility Adapter

Stremio Integration Adapter
```

Every Adapter owns technology.

None own business behaviour.

Anti-Patterns

The following practices are prohibited.

Business Rules In Adapters

Calculating business decisions during mapping.

Domain Imports Infrastructure

Entities importing:

- SQL
- HTTP
- Docker
- SDKs

Fat Adapters

Adapters performing orchestration or business workflows.

Shared Adapters

One Adapter implementing unrelated Ports.

Technology Leaks

Returning infrastructure models directly to the Domain.

Mosaic Guidelines

Within Mosaic:

- Every Adapter MUST implement one or more Ports.
- Adapters MUST own technology-specific code.
- Adapters MUST translate between infrastructure and business concepts.
- Adapters MUST remain thin.
- Adapters MUST NOT contain business rules.
- Infrastructure models MUST NOT cross the Port boundary.
- Adapters SHOULD remain independently replaceable.
- Adapters SHOULD evolve without requiring Domain changes.

Relationship to MEG

Ports define:

> ***What the Domain needs.***

Adapters define:

> ***How those needs are fulfilled.***

The next two chapters separate Adapters into two categories:

- **Driving Adapters**, which bring requests into the Domain.
- **Driven Adapters**, which fulfil the Domain's requests to external systems.

This distinction completes the Ports and Adapters model at the heart of Hexagonal Architecture.

Summary

Adapters are translators.

They isolate technology from the business by converting external representations into Domain concepts and Domain concepts back into external representations.

Within Mosaic, every infrastructure concern exists behind an Adapter.

That simple rule allows databases, APIs, runtimes and transport protocols to evolve continuously while the Domain remains stable, expressive and entirely focused on the business.

Review Status

Status

Draft

Owner

Lead Software Architect

Previous File

`04-driven-ports.md`

Next File

`06-driving-adapters.md`

Driving Adapters

> *Driving Adapters translate external requests into business behaviour. They never become part of the business themselves.*

Purpose

The Domain cannot receive requests directly.

Every interaction with the outside world must first pass through an Adapter.

Examples include:

- HTTP requests
- CLI commands
- Scheduled tasks
- Runtime events
- Extension calls
- Test harnesses

These external interactions differ technologically.

They should not differ architecturally.

Driving Adapters translate these external requests into calls against Driving Ports.

Philosophy

Within Mosaic:

> ***Driving Adapters understand technology. Driving Ports understand the business.***

Driving Adapters should answer one question.

> ***How does this external system communicate with the Domain?***

They should never answer:

> ***What should the business do?***

That responsibility belongs entirely to the Domain.

What Is A Driving Adapter?

A Driving Adapter is an infrastructure component that invokes a Driving Port.

Conceptually:

```
External Actor
```

↓

```
Driving Adapter
↓
Driving Port
↓
Application
↓
Domain
```

The Driving Adapter is responsible for translation.

Nothing more.

External Actors

Many different systems may invoke the same business capability.

Examples include:

```
HTTP
CLI
Scheduler
Runtime Subscriber
Extension
```

Every one becomes a Driving Adapter.

No transport receives special treatment.

Why Driving Adapters Exist

Without Driving Adapters:

```
HTTP
↓
Domain
↓
Business Logic
```

The Domain now understands HTTP.

Instead:

```
HTTP
↓
HTTP Adapter
↓
Driving Port
```

```
↓
Domain
```

Only the Adapter understands HTTP.

The Domain remains completely transport independent.

One Driving Port

Multiple Driving Adapters may invoke the same Driving Port.

Example.

```
PlaybackService
■■■ HTTP Adapter
■■■ CLI Adapter
■■■ Scheduler Adapter
■■■ Runtime Adapter
■■■ Test Adapter
```

The business implementation remains identical.

Only the entry mechanism changes.

Translation

Driving Adapters translate external representations into business requests.

Example.

```
HTTP Request
↓
JSON
↓
Request DTO
↓
ImportMediaRequest
↓
Driving Port
```

Or:

```
CLI Arguments
↓
Business Request
↓
Driving Port
```

Translation ends at the Port boundary.

Validation

Driving Adapters perform transport validation.

Examples include:

- malformed JSON
- missing HTTP headers
- invalid CLI syntax
- protobuf decoding
- authentication tokens

Business validation belongs to the Domain.

The two should never be confused.

Example.

Transport validation.

```
Required JSON Field Missing
```

Business validation.

```
Collection Already Exists
```

Different layers.

Different responsibilities.

Authentication

Authentication belongs to the Driving Adapter layer.

Example.

```
HTTP

↓

Authenticate User

↓

Driving Port
```

The Domain should receive:

```
Authenticated User
```

Not:

```
JWT

↓

Bearer Token
```

```
↓
OAuth Header
```

Security mechanisms remain infrastructure concerns.

Business identity remains a domain concern.

Authorisation

Likewise:

Authorisation decisions should generally occur before entering the Domain.

Example.

```
HTTP
↓
Permission Check
↓
Driving Port
```

The Domain assumes:

The caller has already been authorised to invoke the requested business behaviour.

Business rules remain separate from access control.

Error Translation

Driving Adapters translate Domain errors into transport errors.

Example.

```
Domain
↓
MediaNotFound
↓
HTTP Adapter
↓
404
```

Or:

```
CLI Adapter
↓
Exit Code
```

The Domain should never return:

- HTTP status codes

- CLI exit codes
- GraphQL errors

Adapters perform the translation.

Runtime Events

Within the Reactive Runtime, subscribers become Driving Adapters.

Example.

```
Runtime Event

↓

Subscriber Adapter

↓

RecommendationService

↓

Domain
```

The Domain remains unaware that an Event Bus exists.

Subscribers simply translate runtime events into business requests.

This is one of the key integration points between MEG-002 and MEG-004.

Scheduled Work

Scheduled tasks also become Driving Adapters.

Example.

```
Scheduler

↓

RefreshMetadata()

↓

Driving Port

↓

Domain
```

The Domain should never understand:

- timers
- cron
- scheduling

The scheduler invokes the business through the same contract as every other caller.

Extensions

Extensions invoke the Domain through Driving Adapters.

Example.

```
Extension SDK

↓

Extension Adapter

↓

Driving Port

↓

Domain
```

The Domain remains unaware whether the caller is:

- Core
- Extension
- Test
- CLI

All callers appear identical.

Tests

Tests become Driving Adapters.

Example.

```
Test

↓

Driving Port

↓

Domain
```

No HTTP.

No runtime.

No infrastructure.

The business remains directly testable.

This is one of the greatest practical benefits of Hexagonal Architecture.

Thin Adapters

Driving Adapters SHOULD remain thin.

Typical responsibilities include:

- parsing
- authentication
- transport validation
- mapping
- error translation

They SHOULD NOT contain:

- business rules
- Aggregate manipulation
- orchestration
- persistence

Business behaviour belongs beyond the Port.

Stateless

Driving Adapters SHOULD remain stateless.

They simply translate one request into another.

State belongs to:

- Aggregates
- Repositories
- Runtime

Not the Adapter itself.

Examples Within Mosaic

Examples of Driving Adapters include:

```
REST API

GraphQL API

CLI

Worker Subscriber

Scheduler

Extension Runtime

Integration Tests
```

Each invokes exactly the same business behaviour.

Anti-Patterns

The following practices are prohibited.

Business Rules

Determining business outcomes inside HTTP handlers.

Persistence

Executing SQL directly from Driving Adapters.

Runtime Logic

Managing retries.

Managing scheduling.

Managing workers.

These belong to the runtime.

Aggregate Mutation

Changing Aggregate state directly.

All business behaviour must enter through the Driving Port.

Technology Leakage

Passing:

- HTTP requests
- protobuf messages
- JSON documents

directly into the Domain.

Mosaic Guidelines

Within Mosaic:

- Driving Adapters MUST invoke Driving Ports.
- Driving Adapters MUST remain transport specific.
- Driving Adapters MUST perform translation.
- Driving Adapters MUST perform transport validation.
- Business validation MUST remain inside the Domain.

- Driving Adapters MUST translate Domain errors into transport-specific responses.
- Driving Adapters SHOULD remain stateless.
- Every external caller SHOULD interact through a Driving Adapter.

Relationship to MEG

Driving Ports define:

> ***What business behaviour is available.***

Driving Adapters define:

> ***How external systems invoke that behaviour.***

The next chapter introduces **Driven Adapters**, which perform the opposite role by implementing the capabilities requested by the Domain through Driven Ports.

Together they complete the two halves of the Hexagonal Architecture.

Summary

Driving Adapters isolate the Domain from every external interaction.

Whether requests originate from:

- HTTP
- CLI
- Runtime Events
- Extensions
- Tests

the Domain always receives the same business request through the same Driving Port.

That consistency allows Mosaic to support many interaction models without ever allowing transport concerns to leak into the business itself.

Review Status

Status

Draft

Owner

Lead Software Architect

Previous File

`05-adapters.md`

Next File

07-driven-adapters.md

Driven Adapters

> *Driven Adapters fulfil the Domain's requests by translating business intent into infrastructure operations.*

Purpose

The Domain frequently requires capabilities that only infrastructure can provide.

Examples include:

- persisting Aggregates
- retrieving metadata
- storing artwork
- publishing runtime events
- generating identifiers
- obtaining the current time

The Domain expresses these requirements through Driven Ports.

Driven Adapters satisfy those requirements by implementing the corresponding Port using specific infrastructure technologies.

They form the boundary between the Domain and the external world.

Philosophy

Within Mosaic:

> ***Driven Adapters implement capabilities. They never define them.***

The Domain owns the contract.

The Driven Adapter owns the implementation.

This distinction allows infrastructure to evolve without requiring changes to the Domain Model.

What Is A Driven Adapter?

A Driven Adapter is an infrastructure implementation of a Driven Port.

Conceptually.

```
Domain
↓
Driven Port
↓
Driven Adapter
↓
```

```
Technology
```

The Domain depends only upon the Port.

The Adapter depends upon the technology.

Why Driven Adapters Exist

Without Driven Adapters:

```
Domain

↓

SQL

↓

Database
```

Business behaviour becomes tightly coupled to persistence.

Instead.

```
Domain

↓

PlaybackRepository

↓

PostgreSQL Adapter

↓

Database
```

Only the Adapter understands SQL.

The Domain remains completely independent.

One Port

Every Driven Adapter implements one or more Driven Ports.

Example.

```
MetadataProvider

↓

TMDB Adapter
```

Later.

```
MetadataProvider

↓

AniList Adapter
```

The Domain remains unchanged.

Only the Adapter changes.

Infrastructure Ownership

Driven Adapters own every technology-specific concern.

Examples include:

- SQL
- HTTP clients
- SDKs
- authentication tokens
- connection pools
- file systems
- blob storage
- serialization

None of these concepts belong inside the Domain.

Repository Adapters

Repository implementations are Driven Adapters.

Example.

```
PlaybackRepository
```

↓

```
PostgreSQL Adapter
```

Responsibilities include:

- SQL generation
- transaction management
- persistence mapping
- Aggregate reconstruction

Business behaviour remains inside the Aggregate.

Persistence behaviour remains inside the Adapter.

External Service Adapters

External services should always terminate at a Driven Adapter.

Example.

```
MetadataProvider
```

```
↓
TMDB Adapter
↓
TMDB API
```

The Adapter performs:

- authentication
- request construction
- retries (where appropriate)
- response mapping
- error translation

The Domain receives only business concepts.

Runtime Adapters

The Reactive Runtime is also infrastructure.

Example.

```
Domain Event
↓
RuntimeEventPublisher
↓
Runtime Adapter
↓
Event Bus
```

The Domain remains unaware of:

- workers
- queues
- subscribers
- delivery guarantees

Those concepts belong entirely to the runtime.

Translation

Driven Adapters translate:

```
Business Request
↓
Infrastructure Request
```

and later:

```
Infrastructure Response
↓
Business Result
```

Translation occurs entirely within the Adapter.

Neither the Domain nor the infrastructure should understand one another's models directly.

Mapping

Driven Adapters frequently perform mapping.

Examples include:

```
Aggregate
↓
Database Row
```

```
Database Row
↓
Aggregate
```

```
TMDB Response
↓
Metadata Value Object
```

Mapping should remain symmetrical wherever practical.

Business models should never become persistence models.

Error Translation

Infrastructure errors should never escape the Adapter.

Poor.

```
SQL Error
↓
Domain
```

Preferred.

```
SQL Error
↓
Adapter
↓
MediaNotFound
```

Likewise.

```
HTTP Timeout
↓
Adapter
↓
MetadataUnavailable
```

The Domain should reason about business failures.

Never infrastructure failures.

Configuration

Configuration belongs to infrastructure.

Example.

```
TMDB API Key
↓
Adapter
```

The Domain should never receive:

- API keys
- connection strings
- endpoint URLs

Configuration remains an implementation concern.

Retry Behaviour

Driven Adapters may implement infrastructure retries when interacting with unreliable external systems.

Examples include:

- TMDB
- AniList
- Blob Storage

These retries differ from runtime event retries defined in MEG-002.

Infrastructure retries protect one operation.

Runtime retries protect business workflows.

The two should remain distinct.

Resource Ownership

Driven Adapters own infrastructure resources.

Examples include:

- database connections
- HTTP clients
- blob clients
- caches
- SDK instances

The Domain should never manage resource lifecycles.

Construction and disposal belong to infrastructure.

Composition Root

Driven Adapters are constructed within the Composition Root.

Example.

```
main()
↓
Create PostgreSQL
↓
Create Repository Adapter
↓
Inject Port
↓
Domain
```

The Domain should never instantiate its own Adapters.

Testing

Driven Adapters should be tested independently from the Domain.

Typical tests verify:

- SQL mapping
- HTTP translation
- serialization
- authentication
- protocol compatibility

Domain tests should replace Adapters with simple test implementations.

This separation keeps business tests fast while still validating infrastructure behaviour.

Technology Replacement

Replacing infrastructure should require replacing only the Adapter.

Example.

```
PostgreSQL

↓

CockroachDB
```

or

```
TMDB

↓

AniList
```

The Port remains unchanged.

The Domain remains unchanged.

Only the Adapter evolves.

This is one of the primary reasons Hexagonal Architecture scales well over time.

Multiple Adapters

One Port may have multiple implementations simultaneously.

Example.

```
ArtworkStore

■■■ Local Filesystem

■■■ Blob Storage

■■■ Memory

■■■ Test
```

Selecting the implementation becomes a Composition Root decision.

Not a Domain decision.

Anti-Corruption Layers

Every integration with an external system SHOULD terminate inside a Driven Adapter.

Example.

```
Jellyfin

↓

Jellyfin Adapter

↓

LibraryRepository
```

The Adapter translates:

- terminology
- identifiers
- lifecycle
- business concepts

External models should never leak into the Domain.

Examples Within Mosaic

Examples of Driven Adapters include:

```
PostgreSQL Playback Repository

DuckDB Analytics Repository

TMDB Metadata Provider

AniList Metadata Provider

Blob Artwork Store

Filesystem Artwork Store

Runtime Event Publisher
```

Every Adapter owns implementation.

None own business behaviour.

Anti-Patterns

The following practices are prohibited.

Business Logic

Calculating recommendations inside a repository.

Domain Models Leaking Out

Returning SQL rows directly to the Domain.

Infrastructure Models Leaking In

Passing SDK objects into Aggregates.

Shared Adapters

One Adapter implementing unrelated Ports.

Framework Dependencies

Importing:

- HTTP
- SQL
- Docker

into Domain objects.

Repository As Service

Repositories making business decisions rather than simply persisting Aggregates.

Mosaic Guidelines

Within Mosaic:

- Driven Adapters MUST implement Driven Ports.
- Driven Adapters MUST own technology-specific code.
- Driven Adapters MUST translate infrastructure models into business models.
- Infrastructure errors MUST be translated into business concepts.
- Configuration MUST remain inside infrastructure.
- Domain objects MUST remain infrastructure independent.
- Driven Adapters SHOULD remain independently replaceable.
- Infrastructure changes SHOULD affect only Driven Adapters.

Relationship to MEG

Driving Adapters bring business requests into the Domain.

Driven Adapters fulfil the Domain's external requirements.

Together they complete the Hexagonal Architecture.

```
External System
↓
Driving Adapter
↓
Driving Port
↓
Application
↓
```

```
Domain
↓
Driven Port
↓
Driven Adapter
↓
Infrastructure
```

Every dependency crossing the architectural boundary now passes through an explicit Port and Adapter.

Nothing crosses the boundary accidentally.

Summary

Driven Adapters are where technology finally appears.

They isolate:

- databases
- APIs
- storage
- runtimes
- external services

from the Domain by translating business concepts into infrastructure operations and back again.

Within Mosaic, replacing an infrastructure technology should almost always mean replacing only a Driven Adapter.

If changing a database requires changing an Aggregate, the architectural boundary has been violated.

Review Status

Status

Draft

Owner

Lead Software Architect

Previous File

`06-driving-adapters.md`

Next File

`08-dependency-direction.md`

Dependency Direction

> *Architecture is not defined by layers. It is defined by the direction of dependencies.*

Purpose

The single most important rule within Hexagonal Architecture is:

> ***Dependencies always point towards the Domain.***

Everything else in this specification exists to enforce that rule.

When dependency direction is correct:

- infrastructure becomes replaceable
- business logic remains isolated
- testing becomes simple
- long-term maintenance becomes significantly easier

When dependency direction is violated:

Technology begins shaping the business.

This document defines the dependency rules governing every Mosaic codebase.

Philosophy

Within Mosaic:

> ***The Domain depends upon nothing. Everything else depends upon the Domain.***

This is the architectural centre of the platform.

Every package, interface and dependency should reinforce this rule.

If an engineer is unsure where code belongs, the dependency direction should answer the question.

The Dependency Rule

Every dependency points inward.

```
Infrastructure

↓

Adapters

↓

Ports

↓

Application
```

↓

`Domain`

The reverse direction is prohibited.

The Domain should remain the least coupled part of the platform.

This inward dependency rule is the defining characteristic shared by Hexagonal, Onion and Clean Architecture. [oai_citation:0‡AWS Documentation](https://docs.aws.amazon.com/prescriptive-guidance/latest/hexagonal-architectures/overview.html?utm_source=chatgpt.com)

The Domain

The Domain sits at the centre.

It owns:

- business language
- business rules
- entities
- aggregates
- value objects
- domain services
- domain events

It depends on:

Nothing.

If the Domain imports infrastructure, the architecture has already failed.

The Application Layer

The Application layer coordinates the Domain.

It may depend upon:

- Domain
- Domain Ports

It must not depend upon:

- HTTP
- SQL
- Runtime
- Blob Storage
- Frameworks

The Application layer orchestrates business use cases.

It does not implement infrastructure.

Ports

Ports belong to the Domain or the Application layer.

Ports define:

```
Business Contracts
```

They never depend upon:

```
Adapters
```

Infrastructure implements Ports.

Ports never implement infrastructure.

Ownership remains with the Domain.

Adapters

Adapters sit outside the Hexagon.

They depend upon:

- Ports
- Application
- Domain

They may additionally depend upon:

- HTTP
- SQL
- Docker
- SDKs
- Runtime
- Filesystems

Adapters are allowed to know technology.

The Domain is not.

Infrastructure

Infrastructure is the outermost layer.

Examples include:

- PostgreSQL
- DuckDB

- Blob Storage
- HTTP Servers
- Event Bus
- Docker
- TMDB
- Jellyfin
- Trakt

Infrastructure depends upon everything inside.

Nothing inside depends upon infrastructure.

Allowed Dependencies

The following dependency graph is valid.

```
HTTP Adapter

↓

Application

↓

Domain
```

Likewise.

```
PostgreSQL Adapter

↓

Repository Port

↓

Domain
```

Every dependency points towards the centre.

Forbidden Dependencies

The following dependency graph is prohibited.

```
Domain

↓

HTTP
```

Likewise.

```
Aggregate

↓

SQL
```

Or.

```
Playback
```

```
↓
```

```
Docker
```

Business concepts must never depend upon implementation technologies.

Runtime Dependencies

The Reactive Runtime is infrastructure.

Therefore:

```
Runtime
```

```
↓
```

```
Domain
```

is valid.

```
Domain
```

```
↓
```

```
Runtime
```

is prohibited.

Domain Events should never:

- publish themselves
- understand workers
- understand retries
- understand scheduling

The Runtime adapts to the Domain.

Not the reverse.

Storage Dependencies

Storage technologies remain infrastructure.

Correct.

```
Playback
```

```
↓
```

```
PlaybackRepository
```

```
↓
```

```
PostgreSQL
```

Incorrect.

```
Playback
↓
PostgreSQL
```

The Domain should never import:

- SQL drivers
- ORMs
- database clients

Storage remains entirely outside the Hexagon.

Extension Dependencies

Extensions are infrastructure.

They depend upon:

```
Extension SDK
↓
Ports
↓
Application
↓
Domain
```

The Domain should never know:

- which extensions exist
- how extensions load
- where extensions execute

This allows capabilities to remain stable regardless of platform composition.

Dependency Inversion

Dependency Inversion frequently causes confusion.

Traditional architecture.

```
Business
↓
Database
```

Hexagonal Architecture.

```
Business
↓
```

```
Repository Port
↑
Database Adapter
```

Notice:

Both depend upon the Repository Port.

The Domain owns the abstraction.

Infrastructure implements it.

This inversion allows infrastructure to remain replaceable. [oai_citation:1‡AWS Documentation](https://docs.aws.amazon.com/prescriptive-guidance/latest/cloud-design-patterns/hexagonal-architecture.html?utm_source=chatgpt.com)

Compile-Time Dependencies

Dependency direction is a compile-time concern.

It is determined by:

```
Imports
```

Not:

```
Function Calls
```

A Domain object may indirectly trigger database persistence.

That does not mean it imports the database.

Only imports determine dependency direction.

Runtime Dependencies

Runtime execution frequently flows outward.

Example.

```
HTTP
↓
Domain
↓
Repository
↓
Database
```

This is acceptable.

Execution direction and dependency direction are different concepts.

Execution flows both ways.

Dependencies always point inward.

This distinction is one of the most commonly misunderstood aspects of Hexagonal Architecture.

Package Dependencies

Package imports should naturally reflect architectural direction.

Preferred.

```
internal/

    domain/

    application/

    adapters/

    infrastructure/
```

Dependencies.

```
Adapters

↓

Application

↓

Domain
```

Never:

```
Domain

↓

Adapters
```

The package graph should visually reinforce the architecture.

Cycles

Circular dependencies are prohibited.

Example.

```
Playback

↓

Metadata

↓

Playback
```

If two packages require one another:

One of three things is probably true.

- ownership is unclear
- a Port is missing

- the model requires refinement

Circular dependencies are architectural feedback.

Not compiler inconvenience.

Composition Root

The Composition Root is the only place where:

- concrete implementations
- adapters
- infrastructure

meet the Domain.

Example.

```
main()
↓
Construct Adapter
↓
Inject Port
↓
Application Starts
```

Every other part of the application should remain unaware of concrete implementations.

Testing

Dependency direction naturally enables testing.

```
Test
↓
Fake Adapter
↓
Port
↓
Domain
```

The Domain never knows.

Tests simply provide another Adapter.

This is one of the major practical advantages of the architecture.

Dependency Checklist

Before introducing a dependency ask:

- Does this dependency point inward?
- Does the Domain know about infrastructure?
- Does this import introduce technology into the Domain?
- Does this dependency reinforce or weaken the architecture?
- Could this become a Port instead?

If uncertainty exists:

The dependency probably requires reconsideration.

Anti-Patterns

The following practices are prohibited.

Domain Imports Infrastructure

```
Domain

↓

SQL
```

Runtime Imports

```
Domain

↓

Event Bus
```

Framework Dependencies

Entities importing:

- gin
- echo
- grpc
- ORM libraries

Shared Infrastructure Models

Passing infrastructure models directly into the Domain.

Circular Dependencies

Any package cycle between architectural layers.

Dependency Convenience

Importing infrastructure "because it is easier."

Convenience should never override architecture.

Mosaic Guidelines

Within Mosaic:

- Dependencies MUST always point towards the Domain.
- The Domain MUST remain infrastructure independent.
- Ports MUST be owned by the Domain or Application.
- Adapters MUST implement Ports.
- Infrastructure MUST remain replaceable.
- Runtime MUST remain outside the Domain.
- Package imports MUST reinforce architectural direction.
- Circular dependencies MUST NOT exist.
- The Composition Root MUST assemble the application.

Relationship to MEG

Previous chapters introduced:

- Ports
- Driving Ports
- Driven Ports
- Adapters

This chapter defines the rule connecting them all.

The next chapter introduces the **Composition Root**, the single location where the entire dependency graph is assembled before the application begins execution.

Summary

Every architectural decision within Hexagonal Architecture ultimately reduces to one rule:

> ***Dependencies point inward.***

Everything else:

- Ports
- Adapters
- Dependency Inversion
- Replaceable Infrastructure
- Testability

is simply a consequence of consistently following that principle.

Within Mosaic, preserving dependency direction is one of the primary mechanisms protecting the Domain from the inevitable evolution of technology.

Review Status

Status

Draft

Owner

Lead Software Architect

Previous File

`07-driven-adapters.md`

Next File

`09-composition-root.md`

Composition Root

> *The Composition Root is the only place in the application where concrete implementations know about one another.*

Purpose

Throughout the previous chapters we have established that:

- the Domain owns Ports
- Adapters implement Ports
- dependencies point inward

Eventually, however, the application must actually run.

Concrete implementations must be created.

Dependencies must be connected.

The HTTP server must receive a Service.

Repositories must receive database connections.

The Runtime must receive Event Publishers.

This assembly occurs in one place.

The **Composition Root**.

Philosophy

Within Mosaic:

> ***Construction is centralised. Behaviour is distributed.***

The Composition Root is the only location where:

- infrastructure knows about the Domain
- Adapters know about each other
- concrete implementations are selected
- object graphs are assembled

Every other part of the application remains unaware of concrete implementations.

The Composition Root is a single, logical location where an application's object graph is assembled. [oai_citation:0‡R2 Dev](https://pub-979cff47d4d84105ade2d75c354ef020.r2.dev/Dependency%20Injection%20Principles.pdf?utm_source=chatgpt.com)

What Is A Composition Root?

A Composition Root is the application's entry point.

Typical examples include:

```
cmd/server/main.go

cmd/worker/main.go

cmd/migrate/main.go
```

Every executable has exactly one Composition Root.

Everything else receives dependencies.

Nothing else constructs them.

Why A Composition Root Exists

Without a Composition Root:

```
Playback

↓

Creates Repository

↓

Creates Database

↓

Creates Logger
```

Construction becomes distributed.

Dependencies become hidden.

Testing becomes difficult.

Instead:

```
main()

↓

Create Logger

↓

Create Database

↓

Create Repository

↓

Create Service

↓

Start Application
```

Construction is obvious.

Behaviour remains isolated.

Responsibilities

The Composition Root owns:

- configuration loading
- logger construction
- infrastructure construction
- adapter construction
- dependency wiring
- runtime startup
- graceful shutdown

It intentionally does **not** own:

- business behaviour
- business rules
- orchestration
- validation

Once construction completes, its work is finished.

Dependency Graph

The Composition Root assembles dependencies from the outside in.

```
Configuration

↓

Infrastructure

↓

Driven Adapters

↓

Application

↓

Driving Adapters

↓

Application Starts
```

Notice:

Dependencies are created in the opposite order to dependency direction.

Construction begins outside.

Knowledge still points inward.

Manual Dependency Injection

Within Mosaic, dependencies are assembled explicitly.

Example.

```
[go]
db := postgres.New(...)

repo := postgres.NewPlaybackRepository(db)

service := playback.NewService(repo)

handler := http.NewPlaybackHandler(service)
```

Every dependency is visible.

Nothing is hidden.

Go's explicit construction style naturally complements Hexagonal Architecture.

No Dependency Injection Container

Mosaic intentionally avoids dependency injection containers.

Examples include:

- Spring
- Guice
- Dig
- Wire at runtime

The Composition Root should remain ordinary Go code.

Explicit construction is:

- easier to debug
- easier to understand
- easier to test

The Go ecosystem generally favours explicit wiring over runtime dependency injection containers. [oai_citation:1‡R2 Dev](https://pub-979cff47d4d84105ade2d75c354ef020.r2.dev/Dependency%20Injection%20Principles.pdf?utm_source=chatgpt.com)

Infrastructure Assembly

Infrastructure should be constructed first.

Example.

```
Configuration

↓

Logger
```

```
↓
Database
↓
Blob Storage
↓
HTTP Client
```

These components become dependencies for Adapters.

They should not be created lazily inside business code.

Adapter Assembly

Adapters are constructed after infrastructure.

Example.

```
Database
↓
Playback Repository
↓
Metadata Repository
↓
Collection Repository
```

Each Adapter implements a Port.

The Composition Root decides which implementation to use.

Application Assembly

Application Services receive Ports.

Example.

```
Playback Repository
↓
Playback Service
```

Notice:

The Service knows only about the Port.

It remains unaware of PostgreSQL.

Driving Adapter Assembly

Driving Adapters are constructed last.

Example.

```
Playback Service

↓

HTTP Handler

Playback Service

↓

CLI Command

Playback Service

↓

Runtime Subscriber
```

Every entry point receives the same business capability.

Runtime Assembly

The Reactive Runtime is assembled exactly the same way.

```
Event Publisher

↓

Runtime Adapter

↓

Playback Service
```

or

```
Runtime Subscriber

↓

Recommendation Service
```

The Runtime remains infrastructure.

It is assembled outside the Domain.

Configuration

Configuration belongs exclusively to the Composition Root.

Poor.

```
[go]
os.Getenv(...)
```

inside the Domain.

Preferred.

```
Configuration

↓
```

```
Composition Root
↓
Inject Dependencies
```

Business behaviour should never read environment variables directly.

Environment Selection

The Composition Root selects implementations.

Example.

Development.

```
Filesystem Artwork Store
```

Production.

```
Blob Artwork Store
```

Testing.

```
InMemory Artwork Store
```

The Domain remains unchanged.

Only the Composition Root changes.

Testing

Tests frequently create their own miniature Composition Roots.

Example.

```
Fake Repository
↓
Playback Service
↓
Test
```

Notice:

The test still assembles the dependency graph.

It simply chooses different Adapters.

This is another consequence of explicit dependency construction.

One Composition Root

Each executable should have one Composition Root.

Poor.

```
Service
```

```
↓

Creates Dependencies
```

Good.

```
main()

↓

Everything Constructed
```

Distributed construction eventually becomes a Service Locator.

That architectural pattern is prohibited throughout Mosaic.

Lifecycle Ownership

The Composition Root owns long-lived resources.

Examples include:

- database pools
- HTTP servers
- runtime
- schedulers
- worker pools

It is therefore responsible for:

- startup
- shutdown
- disposal

Ownership should never become ambiguous.

Composition Should Be Boring

Reading `main.go` should feel predictable.

Typical flow.

```
Load Config

↓

Create Infrastructure

↓

Create Adapters

↓

Create Services

↓
```

```
Register Routes
↓
Start Runtime
↓
Wait For Shutdown
```

No surprises.

No hidden construction.

No runtime discovery.

Anti-Patterns

The following practices are prohibited.

Distributed Construction

Creating dependencies throughout the application.

Service Locator

`container.Resolve(...)`

inside business code.

Infrastructure In The Domain

Entities constructing repositories.

Runtime Discovery

Reflection-based dependency resolution during normal execution.

Multiple Composition Roots

One executable assembling dependencies in several unrelated locations.

Hidden Singletons

Global mutable dependencies accessible from anywhere.

Mosaic Guidelines

Within Mosaic:

- Every executable MUST have one Composition Root.

- The Composition Root MUST assemble the complete dependency graph.
- Dependencies MUST be injected explicitly.
- Configuration MUST remain outside the Domain.
- Infrastructure MUST be created before Adapters.
- Adapters MUST be created before Application Services.
- Driving Adapters MUST be assembled last.
- Business code MUST NEVER construct infrastructure.
- Dependency injection containers SHOULD NOT be used.

Relationship to MEG

Previous chapters established:

- Ports
- Adapters
- Dependency Direction

The Composition Root is where those concepts finally become a running application.

The next chapter introduces **Application Services**, the layer responsible for coordinating use cases while preserving the integrity of the Domain Model.

Summary

The Composition Root is one of the simplest files in the application.

It should also be one of the most important.

It answers one question:

> ***How is this application assembled?***

Within Mosaic, the answer should always be visible, explicit and unsurprising.

Once construction completes, the Composition Root steps aside.

The rest of the application simply performs business behaviour.

Review Status

Status

Draft

Owner

Lead Software Architect

Previous File

`08-dependency-direction.md`

Next File

`10-application-services.md`

Application Services

> *Application Services coordinate business behaviour. They never contain it.*

Purpose

The Domain Model defines:

- business rules
- business behaviour
- business invariants

However, external requests frequently require more than invoking a single Aggregate.

Examples include:

- loading an Aggregate
- invoking business behaviour
- persisting changes
- publishing Domain Events

This coordination belongs to the **Application Layer**.

Application Services orchestrate business use cases while ensuring the Domain remains focused exclusively on business decisions.

Philosophy

Within Mosaic:

> ***Application Services coordinate. The Domain decides.***

An Application Service should answer one question.

> ***How should this business use case be executed?***

It should never answer:

> ***What business rule applies?***

Business rules belong to:

- Aggregates
- Entities
- Value Objects
- Domain Services

Never the Application Service.

Application services are widely described as thin orchestration layers that load aggregates, invoke domain behaviour and persist the results without containing domain logic themselves. [oai_citation:0‡Protean

Docs](https://docs.proteanhq.com/concepts/building-blocks/application-services/?utm_source=chatgpt.com)

What Is An Application Service?

An Application Service represents a business use case.

Examples include:

```
Import Media

Resume Playback

Create Collection

Generate Recommendations
```

Each Application Service coordinates work required to fulfil one business objective.

Position Within The Hexagon

Application Services sit immediately outside the Domain.

```
Driving Adapter

↓

Driving Port

↓

Application Service

↓

Domain
```

They form the bridge between:

- infrastructure
- business

Without allowing infrastructure to enter the Domain.

Responsibilities

Application Services are responsible for:

- coordinating use cases
- loading Aggregates
- invoking business behaviour
- persisting Aggregates
- returning business results

They are **not** responsible for:

- business rules

- infrastructure
- transport
- runtime orchestration
- persistence implementation

Responsibilities remain intentionally narrow.

Typical Lifecycle

A typical Application Service follows this sequence.

```
Receive Request

↓

Load Aggregate

↓

Invoke Domain Behaviour

↓

Persist Aggregate

↓

Return Result
```

Notice:

The Application Service performs no business decision making.

It simply coordinates.

Example

Conceptually.

```
Resume Playback

↓

PlaybackRepository.Load()

↓

Playback.Resume()

↓

PlaybackRepository.Save()

↓

Return
```

The business decision:

```
Can playback resume?
```

belongs entirely to:

```
Playback Aggregate
```

Business Logic Belongs Elsewhere

Poor.

```
Application Service

↓

Validate Progress

↓

Calculate Completion

↓

Update Playback
```

Preferred.

```
Application Service

↓

Playback.Resume()
```

The Aggregate performs:

- validation
- invariants
- state transitions
- Domain Events

The Application Service merely invokes it.

Transactions

Application Services frequently define transaction boundaries.

Conceptually.

```
Begin Transaction

↓

Load Aggregate

↓

Business Behaviour

↓

Persist

↓

Commit
```

Transaction management belongs here because it coordinates infrastructure.

It does not belong inside the Domain.

Domain Events

Application Services frequently bridge the Domain and the Runtime.

Example.

```
Playback.Resume()

↓

Domain Event Raised

↓

Persist Aggregate

↓

Runtime Publishes Event
```

Notice:

The Application Service does **not** create the Domain Event.

The Aggregate already did.

It simply ensures the event leaves the Domain after successful persistence.

Ports

Application Services depend only upon Ports.

Example.

```
PlaybackRepository

↓

PlaybackService
```

Never.

```
PostgreSQL

↓

PlaybackService
```

Concrete infrastructure remains invisible.

Stateless

Application Services SHOULD remain stateless.

Poor.

```
PlaybackService
```

```
↓

Current User

↓

Cached Playback

↓

Session
```

Preferred.

```
PlaybackService

↓

Method Parameters

↓

Domain Behaviour

↓

Return
```

State belongs to the Domain.

Not orchestration.

One Use Case Per Method

Every public method SHOULD represent one business use case.

Good.

```
[go]
ResumePlayback(...)

[go]
ArchiveMedia(...)
```

Poor.

```
[go]
Process(...)
```

Methods should reinforce the ubiquitous language.

They should describe outcomes.

Not mechanisms.

Request Models

Application Services may receive request objects.

Example.

```
[go]
type ResumePlaybackRequest struct {

    PlaybackID PlaybackID
```

```
    Position PlaybackPosition
}
```

These models represent business requests.

They should not mirror:

- HTTP payloads
- JSON documents
- protobuf messages

Driving Adapters perform that translation.

Return Values

Application Services should return:

- Aggregates
- Value Objects
- business result objects

They should not return:

- HTTP responses
- JSON
- SQL rows
- infrastructure models

The caller decides how to present the result.

Runtime Independence

Application Services remain unaware of:

- workers
- retries
- scheduling
- event delivery
- subscribers

These concerns belong to the Reactive Runtime.

The Application Service coordinates business behaviour.

Nothing more.

Testing

Application Services should be straightforward to test.

Typical tests verify:

- correct Aggregate loaded
- correct business behaviour invoked
- correct persistence performed

Business rules themselves should be tested at the Domain level.

Application Services verify orchestration.

Not business correctness.

Evolution

Application Services should evolve slowly.

When a method becomes increasingly complicated ask:

> *Has business logic leaked into the Application Service?*

If yes:

Move it:

- into the Aggregate
- into a Domain Service
- into a Value Object

Application Services should become thinner over time.

Not thicker.

Examples Within Mosaic

Examples include:

`ImportMediaService`

`PlaybackService`

`CollectionService`

`RecommendationService`

Each represents:

One business capability.

Each coordinates.

None make business decisions.

Anti-Patterns

The following practices are prohibited.

Business Logic

Calculating business rules inside the Application Service.

Infrastructure Dependencies

Importing:

- SQL
- HTTP
- Docker

directly.

Fat Services

Application Services containing hundreds of lines of business behaviour.

Shared Mutable State

Application Services storing request-specific information.

Aggregate Bypass

Updating persistence directly without invoking the Aggregate.

Runtime Logic

Managing retries.

Managing workers.

Publishing runtime events directly.

These belong elsewhere.

Mosaic Guidelines

Within Mosaic:

- Application Services MUST coordinate business use cases.
- Application Services MUST remain stateless.
- Business rules MUST remain inside the Domain.

- Application Services MUST depend only upon Ports.
- Application Services SHOULD represent one use case per method.
- Request and response models SHOULD remain business focused.
- Domain Events SHOULD originate inside Aggregates.
- Application Services SHOULD remain thin.

Relationship to MEG

The previous chapters established:

- Ports
- Adapters
- Dependency Direction
- Composition Root

Application Services now complete the centre of the Hexagon.

The next chapter explains how the **Reactive Runtime** integrates with Hexagonal Architecture while preserving the Domain's independence.

This is where MEG-002 and MEG-004 finally converge.

Summary

Application Services are frequently misunderstood as another place to write business logic.

Within Mosaic they exist for a much narrower purpose.

They:

- coordinate
- load
- invoke
- persist

The Domain:

- decides
- validates
- protects
- evolves

Keeping these responsibilities separate allows the Domain Model to remain expressive while the Application Layer remains remarkably simple.

Review Status

Status

Draft

Owner

Lead Software Architect

Previous File

`09-composition-root.md`

Next File

`11-runtime-boundary.md`

Runtime Boundary

> *The Runtime executes the business. It is not the business.*

Purpose

The Reactive Runtime introduced in MEG-002 is one of the most sophisticated parts of the Mosaic platform.

It manages:

- event delivery
- scheduling
- retries
- workers
- observability
- lifecycle

Despite this sophistication, the Runtime remains infrastructure.

The Domain must never become aware that it exists.

This document defines the architectural boundary separating the Domain Model from the Reactive Runtime.

Philosophy

Within Mosaic:

> ***The Runtime serves the Domain. The Domain never serves the Runtime.***

The Runtime exists because the Domain requires a mechanism for coordinating work.

The Domain does **not** exist because the Runtime provides one.

This distinction is one of the most important architectural boundaries in the entire platform.

Two Independent Models

Mosaic intentionally maintains two independent models.

```
Business Model

↓

Domain

Execution Model

↓

Reactive Runtime
```

The Business Model describes:

- media
- playback
- metadata
- libraries
- collections

The Execution Model describes:

- workers
- queues
- retries
- scheduling
- event delivery

Neither model should leak into the other.

The Runtime Is Infrastructure

Within Hexagonal Architecture, the Runtime is simply another external system.

Conceptually.

```
HTTP

↓

Runtime

↓

Database

↓

Blob Storage

↓

External APIs
```

All exist outside the Domain.

All communicate through Ports and Adapters.

The Runtime receives no special architectural status simply because it is part of Mosaic.

Business Behaviour

Business behaviour belongs exclusively to the Domain.

Examples include:

```
Playback Completed

Collection Created
```

```
Metadata Corrected
```

The Runtime neither understands nor evaluates these concepts.

It simply transports and coordinates the resulting events.

Runtime Behaviour

The Runtime owns operational concerns.

Examples include:

```
Retry Scheduled

Worker Started

Backpressure Applied

Queue Drained
```

These are runtime concepts.

They are not business concepts.

The Domain should never reference them.

Domain Events

The Domain raises Domain Events.

```
PlaybackSession

↓

Complete()

↓

PlaybackCompleted
```

At this point:

The Domain has finished its work.

It does not:

- publish messages
- schedule retries
- notify subscribers

Those responsibilities begin only after the Domain boundary.

Runtime Translation

A Runtime Adapter bridges the two models.

```
Aggregate

↓
```

```
Domain Event
↓
Runtime Adapter
↓
Runtime Event
↓
Event Bus
```

Notice:

The Domain never imports:

- Event Bus
- Publisher
- Runtime

The Adapter performs the translation.

This keeps the Domain pure while allowing the Runtime to evolve independently.

Subscribers

Runtime Subscribers are Driving Adapters.

Example.

```
Runtime Event
↓
Subscriber Adapter
↓
Driving Port
↓
Application Service
↓
Domain
```

The Domain remains unaware that:

- queues
- workers
- event buses

exist.

It simply receives another business request.

Scheduling

Scheduling belongs entirely to the Runtime.

Poor.

```
[go]
time.Sleep(...)
```

inside an Aggregate.

Preferred.

```
Domain

↓

Request Behaviour

↓

Runtime Scheduler

↓

Driving Adapter

↓

Domain
```

The Runtime owns time.

The Domain owns behaviour.

Retries

Retries are Runtime concerns.

Suppose metadata retrieval fails.

```
Metadata Provider

↓

Failure

↓

Runtime Retry

↓

Later

↓

Driving Adapter

↓

Domain
```

The Domain simply executes business behaviour again.

It remains unaware that this execution resulted from a retry.

This separation keeps business behaviour deterministic while allowing the Runtime to implement sophisticated recovery strategies. [oai_citation:0‡AWS Documentation](https://docs.aws.amazon.com/prescriptive-guidance/latest/cloud-design-patterns/hexagonal-architecture.html?utm_source=chatgpt.com)

Workers

Workers execute Application Services.

They do not execute Aggregates directly.

Conceptually.

```
Worker

↓

Driving Adapter

↓

Application Service

↓

Aggregate
```

The Worker knows:

- scheduling
- cancellation
- retries

The Aggregate knows:

- business rules

Responsibilities remain completely separate.

Cancellation

Runtime cancellation should never leak into business behaviour.

The Runtime decides:

```
Stop Processing
```

The Domain decides:

```
How To Leave Business State Consistent
```

Cancellation is operational.

Consistency is business.

Observability

The Runtime owns:

- traces
- metrics
- queue depth
- worker utilisation
- retry counts

The Domain owns:

- business events
- business identities
- business outcomes

Operational telemetry should never pollute the Domain Model.

Extension Integration

Extensions participate through the Runtime boundary.

Example.

```
Extension

↓

Runtime Event

↓

Driving Adapter

↓

Driving Port

↓

Application

↓

Domain
```

The Domain cannot determine whether the caller is:

- Core
- Extension
- Scheduler
- HTTP

Every caller appears identical.

This greatly simplifies business modelling.

Runtime Replacement

Imagine replacing the current Runtime.

```
Event Bus A

↓

Event Bus B
```

or

```
Worker Engine A

↓

Worker Engine B
```

The Domain should remain unchanged.

Only:

- Runtime Adapters
- Composition Root

require modification.

This is a direct consequence of respecting the Runtime boundary.

Runtime Is Not An Application Service

A common mistake is allowing the Runtime to orchestrate business workflows.

Poor.

```
Runtime

↓

If PlaybackCompleted

↓

Generate Recommendations

↓

Update Statistics

↓

Refresh Metadata
```

The Runtime has now become part of the business.

Instead.

```
PlaybackCompleted

↓

Runtime Delivers

↓
```

```
Independent Subscribers
```

Business behaviour remains inside the Domain.

The Runtime merely coordinates execution.

Runtime APIs

The Domain should never import runtime APIs.

Poor.

```
[go]
runtime.Publish(...)

[go]
runtime.Schedule(...)
```

Preferred.

```
Domain Event

↓

Runtime Adapter

↓

Runtime
```

The Runtime remains replaceable because the Domain does not depend upon its APIs.

Testing

The Runtime boundary makes testing significantly easier.

Domain tests.

```
Application Service

↓

Aggregate

↓

Assertions
```

Runtime tests.

```
Worker

↓

Runtime Adapter

↓

Queues

↓

Retries
```

Each model can be verified independently.

This separation dramatically reduces testing complexity.

Anti-Patterns

The following practices are prohibited.

Runtime Imports

Aggregates importing runtime packages.

Event Bus Inside Domain

Publishing runtime events directly from business objects.

Scheduling Inside Aggregates

Business objects managing timers.

Worker-Aware Business Logic

Business behaviour depending upon worker identity or execution environment.

Retry-Aware Aggregates

Aggregates changing behaviour because an operation is a retry.

Business correctness should remain independent of execution history.

Runtime Orchestration

Runtime components deciding business workflows.

Mosaic Guidelines

Within Mosaic:

- The Runtime MUST remain infrastructure.
- The Domain MUST remain unaware of the Runtime.
- Domain Events MUST cross the Runtime boundary through Adapters.
- Runtime Subscribers MUST invoke Driving Ports.
- Scheduling MUST remain outside the Domain.
- Retries MUST remain outside the Domain.
- Workers MUST execute Application Services rather than business rules.

- Operational telemetry MUST remain outside the Domain.
- Runtime behaviour MUST remain replaceable without modifying business logic.

Relationship to MEG

This chapter completes the integration between:

- **MEG-002 — Reactive Runtime**
- **MEG-003 — Domain-Driven Design**
- **MEG-004 — Hexagonal Architecture**

Together they establish one of the most important architectural principles within Mosaic:

```
Domain

↓

Ports

↓

Adapters

↓

Reactive Runtime

↓

Infrastructure
```

Each layer owns one responsibility.

None understand the implementation details of the next.

Summary

The Runtime is an extraordinarily capable piece of infrastructure.

It schedules.

Retries.

Observes.

Coordinates.

Delivers.

The Domain does none of those things.

It simply models the business.

Maintaining this boundary ensures that the most valuable part of the platform, the business itself, remains protected from the inevitable evolution of the technologies that execute it.

Review Status

Status

Draft

Owner

Lead Software Architect

Previous File

`10-application-services.md`

Next File

`12-testing-the-hexagon.md`

Testing the Hexagon

> *The greatest proof that a Hexagonal Architecture is correct is that the business can be tested without any infrastructure.*

Purpose

One of the primary motivations behind Hexagonal Architecture is testability.

A correctly implemented Hexagon allows the Domain and Application layers to be exercised without requiring:

- PostgreSQL
- DuckDB
- HTTP
- Blob Storage
- Docker
- Event Bus
- External APIs

Testing should become a natural consequence of the architecture.

Not an afterthought.

This document defines how each architectural layer within Mosaic should be tested.

Philosophy

Within Mosaic:

> ***Test business behaviour independently of technology.***

Every architectural boundary exists for two reasons:

- replaceability
- testability

If the Domain requires infrastructure to execute, the boundary has failed.

Testing Strategy

Every layer should be tested independently.

```
Domain

↓

Application

↓

Adapters
```

↓

Integration

↓

End-to-End

Each layer answers different questions.

No layer replaces another.

The Testing Pyramid

Hexagonal Architecture naturally supports the following pyramid.

```
          End-to-End

        Integration Tests

     Adapter Contract Tests

   Application Service Tests

       Domain Tests
```

The majority of tests should exist at the Domain level.

Business logic should receive the highest testing investment.

This isolation is one of the primary benefits of the Ports and Adapters pattern. [oai_citation:0‡AWS Documentation](https://docs.aws.amazon.com/prescriptive-guidance/latest/cloud-design-patterns/hexagonal-architecture.html?utm_source=chatgpt.com)

Domain Tests

Domain tests verify:

- business rules
- Aggregate behaviour
- Value Objects
- Domain Services
- Domain Events
- invariants

Domain tests MUST NOT require:

- databases
- HTTP
- event buses
- runtime
- filesystem

Example.

```
Playback Aggregate
↓
Advance()
↓
PlaybackCompleted Raised
```

The business should execute entirely in memory.

Application Service Tests

Application Services coordinate use cases.

Tests should verify:

- Aggregate loaded
- Domain behaviour invoked
- Aggregate persisted
- Domain Events collected

They should use:

- fake repositories
- fake clocks
- fake identity generators

Infrastructure remains unnecessary.

Adapter Tests

Adapters should be tested independently.

Typical Adapter tests verify:

- request mapping
- response mapping
- SQL generation
- JSON serialization
- API integration
- error translation

Business rules should not appear inside Adapter tests.

The Adapter's responsibility is translation.

Nothing more.

Contract Tests

Every Adapter implements a Port.

Contract tests verify:

```
Port

↓

Every Adapter

↓

Same Behaviour
```

Example.

```
MetadataProvider

■■■ TMDB Adapter ✓

■■■ AniList Adapter ✓

■■■ Fake Adapter ✓
```

Each implementation should satisfy the same behavioural contract.

This ensures infrastructure remains safely replaceable.

Infrastructure Tests

Infrastructure should be tested using real infrastructure.

Examples include:

- PostgreSQL
- DuckDB
- Blob Storage
- Redis
- HTTP APIs

These tests verify:

- configuration
- connectivity
- persistence
- protocol correctness

They do not verify business behaviour.

Runtime Tests

The Reactive Runtime should be tested separately.

Examples include:

- worker execution
- retries
- scheduling
- backpressure
- graceful shutdown
- event delivery

Runtime tests should not re-test business rules.

Business correctness belongs to Domain tests.

End-to-End Tests

End-to-end tests verify complete workflows.

Example.

```
HTTP
↓
Application
↓
Domain
↓
Repository
↓
Runtime
↓
Response
```

These tests provide confidence that architectural boundaries collaborate correctly.

They should remain focused upon important user journeys.

Fake Adapters

The preferred testing strategy is replacing infrastructure with fake Adapters.

Example.

```
PlaybackRepository
↓
InMemoryRepository
Clock
```

↓

FixedClock

MetadataProvider

↓

FakeMetadataProvider

The Domain remains unaware.

Business behaviour remains unchanged.

Test Composition Root

Tests frequently construct a miniature Composition Root.

Example.

Fake Repository

↓

Playback Service

↓

Aggregate

↓

Assertions

This mirrors production assembly.

Only the Adapter implementations differ.

Domain Isolation

A useful architectural test is:

> ***Can the Domain execute with every infrastructure dependency replaced by a fake?***

If the answer is no:

Technology has leaked into the Domain.

The architecture should be reconsidered.

Adapter Isolation

Likewise:

Adapters should be testable independently of business behaviour.

Example.

TMDB Response

↓

```
Metadata Adapter
↓
Metadata Value Object
```

No Aggregate behaviour should be required.

Error Testing

Errors should be tested at the correct layer.

Domain.

```
Invalid Business Rule
```

Adapter.

```
HTTP Timeout
```

Runtime.

```
Retry Exhausted
```

Every layer owns its own failure semantics.

Mutation Safety

Domain tests should verify that invariants cannot be violated.

Example.

```
Progress > Duration
↓
Rejected
```

Testing invalid business state is often more valuable than testing successful behaviour.

Event Testing

Domain tests verify:

```
PlaybackCompleted Raised
```

Runtime tests verify:

```
PlaybackCompleted Delivered
```

These are different concerns.

They should remain independently testable.

Mocking

Within Mosaic:

Prefer:

- fake repositories
- in-memory implementations
- deterministic adapters

Avoid excessive mocking frameworks.

Hexagonal Architecture naturally reduces the need for complex mocks because Ports already provide clean substitution points. [oai_citation:1‡AWS Documentation](https://docs.aws.amazon.com/prescriptive-guidance/latest/cloud-design-patterns/hexagonal-architecture.html?utm_source=chatgpt.com)

Test Data

Business test data should use business terminology.

Good.

```
Library
```

```
Playback
```

```
Collection
```

Poor.

```
Row1
```

```
ObjectA
```

```
DTO2
```

The ubiquitous language should remain consistent inside tests.

Tests are executable documentation.

Architecture Verification

A useful question is:

> ***Can I delete every Adapter and still execute the Domain?***

If yes:

The Hexagon has been preserved.

If no:

Infrastructure has become coupled to business behaviour.

Anti-Patterns

The following practices are prohibited.

Domain Tests Using Databases

HTTP Required For Business Tests

Runtime Required For Aggregate Tests

SQL Assertions Inside Domain Tests

Infrastructure Models Inside Domain Tests

Business Logic Inside Adapter Tests

Mosaic Guidelines

Within Mosaic:

- Domain tests MUST remain infrastructure independent.
- Application Services SHOULD use fake Ports.
- Adapters SHOULD be tested independently.
- Contract tests SHOULD verify Port implementations.
- Runtime SHOULD be tested separately from business logic.
- End-to-end tests SHOULD validate complete workflows.
- Fake Adapters SHOULD be preferred over complex mocks.
- Every architectural layer SHOULD own its own tests.

Relationship to MEG

Hexagonal Architecture exists partly to make testing a natural property of the design.

The previous chapters explained:

- dependency direction
- Ports
- Adapters
- Composition

This chapter demonstrates the practical result.

Good architecture naturally produces good tests.

The next chapter provides practical modelling guidance for engineers implementing Hexagonal Architecture throughout the Mosaic platform.

Summary

Testing is one of the strongest indicators of architectural quality.

Within Mosaic:

- the Domain should run without infrastructure
- Application Services should coordinate without technology
- Adapters should be independently replaceable
- the Runtime should be independently verifiable

When every layer can be tested in isolation, the architecture has successfully separated business from technology.

That is the real promise of Hexagonal Architecture.

Review Status

Status

Draft

Owner

Lead Software Architect

Previous File

`11-runtime-boundary.md`

Next File

`13-modelling-guidelines.md`

Modelling Guidelines

> *Every architectural decision should strengthen the boundary between the business and the outside world.*

Purpose

The previous chapters introduced the structural building blocks of Hexagonal Architecture:

- Ports
- Adapters
- Dependency Direction
- Composition Root
- Application Services
- Runtime Boundaries

This document brings those concepts together into practical guidance for engineers implementing new capabilities within the Mosaic platform.

Its purpose is to answer one question.

> ***"Where should this code actually live?"***

Philosophy

Within Mosaic:

> ***Model the business first. Connect it to technology afterwards.***

When designing a new capability:

1. Discover the business. 2. Model the Domain. 3. Define the Ports. 4. Implement the Adapters. 5. Assemble everything in the Composition Root.

Never reverse this order.

Start With The Domain

Every feature should begin inside the Domain.

Ask:

- What business capability exists?
- Which Aggregate owns it?
- Which business rules apply?
- Which Domain Events occur?

Do **not** begin with:

- database schema
- HTTP endpoints
- REST APIs
- runtime events

Technology follows the Domain.

Never the reverse.

Define Ports Last

One of the most common mistakes is designing Ports before understanding the Domain.

Instead:

```
Business Behaviour
↓
Domain Model
↓
Port
↓
Adapter
```

Ports should emerge naturally from business requirements.

Not hypothetical infrastructure.

One Dependency Rule

Whenever adding a dependency ask:

> ***Does this dependency point towards the Domain?***

If yes:

Proceed.

If no:

The dependency probably belongs elsewhere.

Dependency direction should answer more architectural questions than folder structure.

Business Before Infrastructure

Suppose a new feature requires TMDB.

Incorrect thought process.

```
TMDB API
```

↓

How do I use it?

Preferred.

Metadata

↓

What business capability do I need?

↓

MetadataProvider

↓

TMDB Adapter

The business requirement comes first.

The infrastructure follows.

Choosing A Port

Before introducing a Port ask:

- Does the Domain genuinely depend upon this capability?
- Is this a business requirement?
- Can another Port already satisfy this behaviour?

Avoid creating Ports because:

> *"We might need another implementation later."*

Ports exist because the Domain requires stable contracts.

Not speculative flexibility.

AWS recommends introducing ports where the business genuinely requires external interaction, rather than creating unnecessary abstraction layers. [oai_citation:0‡AWS Documentation](https://docs.aws.amazon.com/prescriptive-guidance/latest/hexagonal-architectures/best-practices.html?utm_source=chatgpt.com)

Choosing An Adapter

Every Adapter should answer:

> ***Which technology am I isolating?***

Examples.

PostgreSQL

TMDB

Filesystem

HTTP

If an Adapter cannot answer that question clearly, its responsibility should be reconsidered.

One Adapter, One Concern

Adapters should remain cohesive.

Good.

```
TMDB Adapter
```

Poor.

```
ExternalServicesAdapter

↓

TMDB

AniList

Trakt

Jellyfin

Docker
```

Technology boundaries should remain explicit.

Keep The Domain Pure

The following should never appear inside the Domain.

- SQL
- HTTP
- JSON
- Docker
- Runtime
- Logging
- Environment variables
- Framework annotations

If these concepts appear:

The boundary has already been crossed.

Keep Application Services Thin

Application Services should follow the same structure.

```
Receive Request

↓

Load Aggregate

↓
```

```
Invoke Behaviour

↓

Persist Aggregate

↓

Return Result
```

If additional business logic appears:

Move it into:

- Aggregate
- Entity
- Value Object
- Domain Service

Application Services coordinate.

They do not decide.

Translate At The Boundary

Every translation should occur at a boundary.

Examples.

```
JSON

↓

Request DTO

↓

Business Request
```

```
Aggregate

↓

Database Row
```

```
TMDB Response

↓

Metadata
```

Translation should never occur inside the Domain.

Prefer Explicit Mapping

Avoid magical mapping libraries where they obscure intent.

Good.

```
[go]
Metadata{
```

```
    Title: response.Title,
    Year:  response.Year,
}
```

Poor.

```
[go]
mapper.Map(...)
```

Explicit mapping is usually:

- easier to debug
- easier to review
- easier to evolve

Clarity is generally more valuable than reducing a few lines of code.

Infrastructure Should Be Replaceable

Ask:

> ***Could I replace this technology without modifying the Domain?***

Examples.

```
PostgreSQL

↓

CockroachDB

TMDB

↓

AniList

REST

↓

GraphQL
```

If the answer is "no":

Technology has probably leaked through the boundary.

Design For Testing

A useful architectural question is:

> ***Can I test this without infrastructure?***

If the answer is yes:

The boundary is probably correct.

If the answer is no:

Determine which dependency has entered the wrong layer.

Runtime Is Infrastructure

One subtle guideline deserves repeating.

The Reactive Runtime belongs outside the Hexagon.

The Domain should never know:

- workers
- retries
- scheduling
- queues
- event buses

Those concepts remain infrastructure.

Business behaviour remains pure.

Folder Structure Follows Architecture

Packages should reflect architectural ownership.

Example.

```
internal/

    domain/

    application/

    adapters/

        http/

        postgres/

        tmdb/

        runtime/

    bootstrap/
```

The folder structure should communicate:

- dependency direction
- ownership
- boundaries

It should never encourage architectural violations.

A clear separation between domain, entry points and adapters is a common recommendation for Ports and Adapters implementations. [oai_citation:1‡AWS Documentation](https://docs.aws.amazon.com/prescriptive-guidance/latest/hexagonal-architectures/best-practices.html?utm_source=chatgpt.com)

Refactor Towards The Hexagon

Existing code rarely starts perfectly.

When refactoring ask:

- Can this dependency move outward?
- Can this behaviour move inward?
- Can this translation occur at the boundary?
- Can this Port become smaller?
- Can this Adapter become simpler?

The Hexagon should become more explicit over time.

Not less.

Architecture Checklist

Before merging a new capability confirm:

- [ ] Business behaviour lives in the Domain.
- [ ] Dependencies point inward.
- [ ] Ports describe business capabilities.
- [ ] Adapters isolate technology.
- [ ] Application Services remain thin.
- [ ] Runtime concerns remain outside the Domain.
- [ ] Infrastructure remains replaceable.
- [ ] Tests can execute without infrastructure.
- [ ] The Composition Root assembles all concrete implementations.

Common Modelling Mistakes

Avoid:

- designing around frameworks
- exposing infrastructure through Ports
- placing business rules in Adapters
- generic "manager" classes
- large Ports
- large Adapters
- Domain imports of infrastructure packages

Most architectural problems begin as small convenience decisions.

Mosaic Guidelines

Within Mosaic:

- The Domain MUST be designed before infrastructure.
- Ports SHOULD emerge naturally from business requirements.
- Adapters MUST isolate technology.
- Translation MUST occur at architectural boundaries.
- Application Services SHOULD remain orchestration only.
- Runtime concerns MUST remain outside the Domain.
- Infrastructure MUST remain replaceable.
- Architectural simplicity SHOULD always outweigh unnecessary abstraction.

Relationship to MEG

This chapter completes the practical implementation guidance of MEG-004.

The remaining documents describe:

- architectural reasoning (ADRs)
- contributor expectations
- terminology
- references

Together, MEG-001 through MEG-004 now define:

- how software is written
- how it executes
- how the business is modelled
- how technology is prevented from corrupting that model

Summary

Hexagonal Architecture is ultimately about one idea.

> ***Protect the business from technology.***

Every Port.

Every Adapter.

Every dependency.

Every package.

Every layer.

Should reinforce that principle.

When implemented consistently, changing technologies becomes routine.

Changing the business remains deliberate.

That is the architectural foundation upon which the rest of the Mosaic platform is built.

Review Status

Status

Draft

Owner

Lead Software Architect

Previous File

`12-testing-the-hexagon.md`

Next File

`14-adrs.md`

Architectural Decision Records (ADRs)

> *Architectural boundaries are among the most expensive decisions to change. They should never exist without recorded reasoning.*

Purpose

Hexagonal Architecture defines how the Mosaic platform is structured.

It determines:

- dependency direction
- ownership
- technology boundaries
- integration strategy
- long-term maintainability

Changes to these principles affect every capability built upon the platform.

Architectural Decision Records (ADRs) preserve the reasoning behind those decisions so that future contributors understand not only *what* the architecture is, but *why* it exists.

Philosophy

Within Mosaic:

> ***Architectural boundaries should be intentional, stable and historically traceable.***

The Hexagon should evolve because our understanding improves.

It should never evolve because implementation became convenient.

Why Hexagonal ADRs Matter

Infrastructure changes frequently.

Architecture should not.

Without ADRs, engineers eventually ask:

- Why are Ports owned by the Domain?
- Why are Runtime Events outside the Hexagon?
- Why don't repositories return database models?
- Why are extensions treated as infrastructure?

Without documented reasoning, those discussions repeat.

ADRs preserve architectural knowledge.

When An ADR Is Required

A Hexagonal ADR SHOULD be created whenever a decision changes:

- dependency direction
- Port ownership
- Adapter responsibilities
- Composition Root behaviour
- Application Service responsibilities
- Runtime boundaries
- integration strategy
- infrastructure boundaries

If the decision changes how the architecture is structured, it deserves an ADR.

Examples

Examples of Hexagonal ADRs include:

```
ADR-001

Hexagonal Architecture

ADR-002

Domain Owns Ports

ADR-003

Reactive Runtime Outside The Hexagon

ADR-004

Application Service Responsibilities

ADR-005

Repository Ownership

ADR-006

Extension Integration Boundary

ADR-007

Composition Root Strategy
```

These decisions influence every implementation within the platform.

Architectural Stability

Hexagonal Architecture should evolve conservatively.

Changing:

```
HTTP Framework
```

is inexpensive.

Changing:

```
Dependency Direction
```

affects:

- every package
- every Port
- every Adapter
- every test
- every extension

Architectural stability should therefore be prioritised.

ADR Structure

Every Hexagonal ADR SHOULD contain:

```
Title

↓

Status

↓

Context

↓

Architectural Problem

↓

Options

↓

Decision

↓

Consequences

↓

Migration

↓

Related Specifications
```

Migration guidance is particularly important because architectural changes frequently affect many repositories.

Context

The Context section should describe:

- current architecture
- dependency relationships
- technical constraints
- business requirements

Readers unfamiliar with Mosaic should understand why the architectural discussion became necessary.

Architectural Problem

The problem statement should describe architecture.

Good.

```
The Domain currently depends upon runtime infrastructure.
```

Poor.

```
We should refactor some packages.
```

The problem should remain understandable independently of implementation.

Options

Every significant architectural decision SHOULD evaluate alternatives.

Example.

```
Layered Architecture

Hexagonal Architecture

Clean Architecture
```

Each option should document:

- benefits
- drawbacks
- operational impact
- maintenance implications

Rejected alternatives remain valuable engineering knowledge.

Decision

The Decision section answers one question.

> ***Which architectural approach becomes the new standard?***

Implementation details belong elsewhere.

The ADR records the decision itself.

Consequences

Every architectural decision introduces trade-offs.

Example.

Choosing:

```
Domain Owns Ports
```

Benefits.

- infrastructure independence
- testability
- replaceable adapters

Costs.

- additional interfaces
- more explicit composition
- initial learning curve

Trade-offs should always be documented honestly.

No architecture is free.

Migration

Architectural changes often require gradual migration.

Migration guidance should explain:

- affected repositories
- dependency changes
- compatibility expectations
- rollout strategy

Large-scale architectural changes should never rely solely on implementation.

Runtime Integration

Changes affecting the Runtime boundary SHOULD receive particular scrutiny.

Example.

```
Runtime
```

↓

```
Outside Hexagon
```

Moving this boundary affects:

- MEG-002

- MEG-003
- every Adapter
- every Application Service

Cross-specification changes should therefore be documented explicitly.

Cross References

Hexagonal ADRs SHOULD reference:

- related MEG specifications
- relevant runtime ADRs
- domain ADRs
- implementation repositories

Architecture should remain navigable.

Knowledge should not become fragmented.

Repository Structure

Recommended layout.

```
architecture/

    adrs/

        ADR-001-hexagonal-architecture.md

        ADR-002-domain-owns-ports.md

        ADR-003-runtime-boundary.md

        ADR-004-composition-root.md

        ADR-005-extension-boundary.md
```

Architectural decisions should remain close to the specifications they influence.

Review Process

Hexagonal ADRs SHOULD receive architectural review.

Review should consider:

- dependency direction
- coupling
- maintainability
- testability
- runtime integration

- long-term evolution

Architecture should lead implementation.

Not follow it.

Documentation

Accepted Hexagonal ADRs SHOULD eventually be reflected within:

- MEG specifications
- package documentation
- architecture diagrams
- contributor guidance

The written architecture should always reflect the implemented architecture.

Mosaic Guidelines

Within Mosaic:

- Significant architectural boundary changes SHOULD have ADRs.
- Dependency direction changes MUST be documented.
- Alternatives SHOULD be evaluated explicitly.
- Architectural trade-offs MUST be acknowledged.
- Migration guidance SHOULD accompany major architectural changes.
- Historical ADRs MUST remain available.
- The implemented architecture SHOULD always be traceable back to documented architectural decisions.

Relationship to MEG

MEG-004 defines:

> ***How the architecture is structured today.***

Hexagonal ADRs explain:

> ***Why the architecture is structured that way.***

Together they preserve the architectural intent of the platform.

Future contributors should inherit both the design and the reasoning behind it.

Summary

Hexagonal Architecture protects the Domain from technology.

Architectural Decision Records protect that architecture from time.

Within Mosaic, the architecture should evolve through deliberate engineering judgement rather than accidental implementation.

Every significant boundary should therefore exist because somebody consciously chose it, documented it and accepted its trade-offs.

Review Status

Status

Draft

Owner

Lead Software Architect

Previous File

`13-modelling-guidelines.md`

Next File

`15-contributor-guidance.md`

Contributor Guidance

> *Every contribution either strengthens or weakens the architectural boundary. There is no neutral change.*

Purpose

The Hexagonal Architecture protects the most valuable part of the Mosaic platform:

> ***The Domain Model.***

Every contributor therefore shares responsibility for preserving:

- dependency direction
- technology independence
- architectural boundaries
- replaceable infrastructure

This document provides practical guidance for engineers implementing new capabilities within the Hexagonal Architecture.

Philosophy

Within Mosaic:

> ***Protect the Domain before adding infrastructure.***

Business behaviour should remain stable.

Infrastructure should remain replaceable.

Whenever these goals conflict:

The Domain wins.

Always.

Before Writing Code

Before implementing a new feature ask:

- Which Bounded Context owns this?
- Which Aggregate owns the behaviour?
- Does the Domain already model this concept?
- Is infrastructure influencing the design?

If implementation discussions begin before business modelling:

Return to MEG-003.

The Domain should always lead implementation.

Before Creating A Port

Ask:

- Does the Domain genuinely require this capability?
- Is this describing business behaviour?
- Could an existing Port already satisfy this requirement?

Avoid introducing Ports simply because:

> *"We might need another implementation."*

Ports should emerge from business requirements.

Not speculation.

Before Creating An Adapter

Ask:

- Which technology am I isolating?
- Does this Adapter perform translation?
- Does any business behaviour appear here?

If the Adapter contains business rules:

The architecture has already begun drifting.

Business behaviour belongs inside the Domain.

Before Adding A Dependency

Ask one question.

> ***Does this dependency point inward?***

If yes:

Proceed.

If no:

Reconsider the design.

Dependency direction is one of the strongest architectural indicators available.

Before Importing A Package

Every import should reinforce the Hexagon.

Allowed.

```
Adapter
```

↓

```
Application
```

↓

```
Domain
```

Prohibited.

```
Domain
```

↓

```
Infrastructure
```

The compiler should naturally reinforce the architecture.

Imports should tell the same architectural story as the documentation.

Before Adding Infrastructure

Ask:

- Does the Domain already expose a Port?
- Should infrastructure implement that Port?
- Does the Domain actually require this capability?

Infrastructure should adapt itself to existing Ports whenever practical.

Avoid changing the Domain solely because a new technology has been introduced.

Before Modifying A Port

Ports are long-lived contracts.

Changing one affects:

- the Domain
- every Adapter
- every test
- every extension

Before modifying a Port ask:

- Is the business changing?
- Or only the infrastructure?

If only infrastructure changes:

The Port probably should not.

Before Modifying An Adapter

Adapters should evolve freely.

Examples include:

- SQL optimisation
- API version upgrades
- SDK replacement
- protocol changes

These changes should rarely affect the Domain.

If they do:

Review the architectural boundary.

Before Creating An Application Service

Application Services should coordinate.

Not decide.

Ask:

- Am I loading an Aggregate?
- Am I invoking business behaviour?
- Am I persisting changes?

If additional business logic appears:

Move it into:

- Aggregate
- Entity
- Value Object
- Domain Service

The Application layer should remain intentionally thin.

Before Introducing Runtime Behaviour

The Reactive Runtime is infrastructure.

Ask:

- Does the Domain need to know this?
- Or should an Adapter translate it?

Examples.

Poor.

```
Aggregate

↓

Publish Runtime Event
```

Preferred.

```
Aggregate

↓

Raise Domain Event

↓

Runtime Adapter

↓

Runtime Event
```

MEG-002 and MEG-004 should reinforce one another.

Never compete.

Before Merging

Every architectural contribution SHOULD satisfy the following checklist.

Domain

- Business behaviour remains inside the Domain.
- No infrastructure packages imported.
- Ubiquitous Language remains consistent.

Ports

- Ports describe business capabilities.
- Ports remain technology independent.
- Ports remain focused.

Adapters

- Technology isolated.
- Translation only.
- No business rules.

Dependencies

- Dependencies point inward.

- No circular imports.
- Infrastructure remains replaceable.

Runtime

- Runtime remains outside the Domain.
- Domain Events remain infrastructure independent.
- Runtime integration occurs through Adapters.

Documentation

- MEG updated if required.
- ADR created where appropriate.
- Diagrams remain accurate.
- Examples remain consistent.

Architecture documentation should evolve alongside implementation.

Recognising Architectural Drift

The following symptoms usually indicate the Hexagon is weakening.

- SQL inside the Domain.
- HTTP types crossing Port boundaries.
- Runtime APIs imported by Aggregates.
- Growing Application Services.
- Business logic inside Adapters.
- Infrastructure exceptions leaking into the Domain.

Architectural drift should be corrected early.

Small violations accumulate quickly.

Refactoring Towards The Hexagon

When improving existing code ask:

- Can this dependency move outward?
- Can this translation move into an Adapter?
- Can this business rule move into an Aggregate?
- Can this Port become smaller?
- Can this technology disappear behind a Port?

Refactoring should make boundaries more explicit.

Not blur them.

Review Mindset

Architecture reviews should ask:

- Does this strengthen dependency direction?
- Does this improve replaceability?
- Does this reduce coupling?
- Does the Domain remain pure?
- Would replacing this technology require Domain changes?

These questions are generally more valuable than debating implementation style.

Learning The Architecture

New contributors SHOULD study MEG-004 in the following order.

```
Hexagonal Philosophy

↓

Ports

↓

Adapters

↓

Dependency Direction

↓

Composition Root

↓

Application Services

↓

Runtime Boundary
```

Understanding dependency direction first makes every later concept significantly easier to understand.

Engineering Culture

Contributors should strive to:

- simplify dependencies
- reduce coupling

- improve naming
- remove technology leakage
- preserve Domain purity
- question unnecessary abstraction

The architecture should become more obvious over time.

Not more clever.

Contributor Checklist

Before requesting review, confirm:

- [ ] The Domain remains technology independent.
- [ ] Dependencies point inward.
- [ ] Ports describe business capabilities.
- [ ] Adapters isolate technology.
- [ ] Application Services remain orchestration only.
- [ ] Runtime concerns remain outside the Domain.
- [ ] Infrastructure remains replaceable.
- [ ] Documentation has been updated.
- [ ] The architecture is simpler or clearer than before.

Relationship to MEG

This document explains how contributors should evolve the Hexagonal Architecture established throughout MEG-004.

The previous chapters define:

> ***How the architecture should be structured.***

This chapter defines:

> ***How engineers should preserve that structure over time.***

Architecture survives not because diagrams exist.

It survives because contributors consistently reinforce its principles.

Summary

Hexagonal Architecture is not maintained through frameworks.

It is maintained through engineering discipline.

Every contributor influences whether the Domain remains:

- independent
- testable
- replaceable
- understandable

Within Mosaic, every change should strengthen the architectural boundary between the business and technology.

Because once that boundary begins to erode, the cost of every future change begins to increase with it.

Review Status

Status

Draft

Owner

Lead Software Architect

Previous File

`14-adrs.md`

Next File

`glossary.md`

Glossary

> *Architecture succeeds when every engineer means the same thing by the same word.*

Purpose

This glossary defines the terminology used throughout the Mosaic Hexagonal Architecture specification.

These definitions establish the canonical architectural vocabulary for:

- Architecture Specifications
- ADRs
- Source Code
- Documentation
- Extension SDKs
- Engineering Discussions

Where a term has a specific meaning within the Mosaic Architecture, that definition takes precedence over informal usage.

A

Adapter

An infrastructure component that translates between the Domain and an external technology.

Adapters implement Ports.

They own:

- translation
- protocol conversion
- infrastructure integration

They do **not** own business behaviour.

Application Layer

The layer immediately outside the Domain responsible for coordinating business use cases.

Application Services belong to this layer.

The Application Layer orchestrates.

The Domain decides.

Application Service

A component responsible for coordinating a business use case.

Application Services:

- load Aggregates
- invoke business behaviour
- persist changes

They do not implement business rules.

C

Composition Root

The single location where:

- infrastructure
- adapters
- application services
- runtime

are assembled into a running application.

Typically:

`cmd/server/main.go`

The Composition Root owns dependency construction.

D

Dependency Direction

The architectural rule stating that every dependency points towards the Domain.

Infrastructure depends upon the Domain.

The Domain depends upon nothing external.

Domain

The business core of the application.

The Domain owns:

- Entities
- Aggregates
- Value Objects

- Domain Services
- Domain Events

The Domain remains completely independent of infrastructure.

Driven Adapter

An Adapter implementing a Driven Port.

Examples include:

- PostgreSQL Repository
- TMDB Metadata Provider
- Blob Storage Adapter

Driven Adapters fulfil Domain requests.

Driven Port

A Port describing a capability required by the Domain.

Examples include:

- Repository
- Metadata Provider
- Artwork Store

The Domain owns these contracts.

Infrastructure implements them.

Driving Adapter

An Adapter translating external requests into Domain behaviour.

Examples include:

- HTTP Controllers
- CLI Commands
- Runtime Subscribers
- Scheduler Tasks

Driving Adapters invoke Driving Ports.

Driving Port

A contract describing business behaviour exposed by the Application.

Examples include:

- Resume Playback

- Import Media
- Create Collection

Driving Ports define use cases.

Not transport protocols.

H

Hexagon

The conceptual boundary surrounding the Domain and Application.

Everything inside the Hexagon represents business behaviour.

Everything outside represents infrastructure.

The hexagon is a visual metaphor rather than a structural requirement. [oai_citation:0‡Alistair Cockburn](https://alistair.cockburn.us/hexagonal-architecture?utm_source=chatgpt.com)

Hexagonal Architecture

An architectural style introduced by Alistair Cockburn.

Also known as:

- Ports and Adapters

Its central principle is:

> *Dependencies point inward.*

Business behaviour remains isolated from external technology.

[oai_citation:1‡Alistair Cockburn](https://alistair.cockburn.us/hexagonal-architecture?utm_source=chatgpt.com)

I

Infrastructure

Everything outside the Domain.

Examples include:

- HTTP
- PostgreSQL
- Blob Storage
- Docker
- Runtime
- External APIs

Infrastructure adapts itself to the Domain.

Never the reverse.

P

Port

A business contract owned by the Domain or Application Layer.

Ports define:

- required capabilities
- exposed behaviour

Ports never define implementation.

R

Replaceability

The architectural property whereby one technology may be exchanged for another without modifying business behaviour.

Replaceability is achieved through:

- Ports
- Adapters
- Dependency Inversion

Runtime Boundary

The architectural separation between:

- the Domain Model
- the Reactive Runtime

The Runtime coordinates execution.

The Domain models business behaviour.

T

Technology Leakage

The accidental introduction of infrastructure concepts into the Domain.

Examples include:

- SQL

- HTTP
- JSON
- Docker
- Runtime APIs

Technology leakage violates Hexagonal Architecture.

Translation

The process performed by Adapters to convert between:

- infrastructure models
- business models

Translation always occurs at the architectural boundary.

Never inside the Domain.

U

Use Case

A business operation exposed through a Driving Port.

Examples include:

- Import Media
- Resume Playback
- Archive Collection

Application Services coordinate use cases.

Aggregates implement business behaviour.

Common Acronyms

Acronym	Meaning
ACL	Anti-Corruption Layer
ADR	Architectural Decision Record
API	Application Programming Interface
DTO	Data Transfer Object
HTTP	Hypertext Transfer Protocol
MEG	Mosaic Engineering Guidelines
SDK	Software Development Kit
SQL	Structured Query Language
UI	User Interface

Relationship to MEG-004

This glossary supports every document within the Hexagonal Architecture specification.

Definitions should remain consistent across:

- Architecture Specifications
- Runtime Documentation
- Domain Documentation
- Extension SDKs
- Contributor Guidance

Whenever architectural terminology evolves, this glossary SHOULD be updated before introducing new terminology elsewhere.

Review Status

Status

Draft

Owner

Lead Software Architect

Previous File

`15-contributor-guidance.md`

Next File

`references.md`

References

> *Hexagonal Architecture is not a framework. It is a way of protecting business knowledge from technological change.*

Purpose

This document records the primary references that informed the architectural principles established throughout MEG-004.

Hexagonal Architecture has existed for over two decades.

The Mosaic implementation builds upon those established ideas while adapting them to:

- Event-Driven Runtime
- Domain-Driven Design
- Go Engineering
- Extension-first architecture
- Long-lived software platforms

The purpose of these references is to explain the architectural lineage of the Mosaic platform rather than prescribe implementation.

Primary References

Hexagonal Architecture (Ports & Adapters)

Author

Alistair Cockburn

Purpose

The original paper introducing:

- Ports
- Adapters
- Technology independence
- Dependency inversion
- Testable architecture

This paper forms the primary architectural foundation of MEG-004.

The central architectural principle adopted by Mosaic is:

> ***The application communicates through Ports while Adapters isolate external technologies.***
> *[oai_citation:0‡Alistair Cockburn](https://alistair.cockburn.us/hexagonal-architecture?utm_source=chatgpt.com)*

URL

https://alistair.cockburn.us/hexagonal-architecture/

Hexagonal Architecture Explained

Authors

Alistair Cockburn

Juan Manuel Garrido de Paz

Purpose

A modern expansion of the original Ports & Adapters architecture including practical implementation guidance.

Recommended reading for contributors wishing to understand the evolution of Hexagonal Architecture beyond the original article. [oai_citation:1‡Google Books](https://books.google.com/books/about/Hexagonal_Architecture_Explained.html?id=Eim20AEACAAJ&utm_source=chatgpt.com)

Supporting Architecture References

AWS Prescriptive Guidance

Purpose

Practical guidance for implementing Hexagonal Architecture within modern cloud-native systems.

Topics include:

- dependency isolation
- testability
- replaceable infrastructure
- technology independence

The AWS guidance aligns closely with Mosaic's goal of protecting the Domain from infrastructure concerns. [oai_citation:2‡AWS Documentation](https://docs.aws.amazon.com/prescriptive-guidance/latest/cloud-design-patterns/hexagonal-architecture.html?utm_source=chatgpt.com)

Dependency Injection Principles, Practices and Patterns

Author

Mark Seemann

Topics

- Composition Root
- Dependency Injection
- Dependency Inversion
- object composition

This work strongly influenced:

- explicit composition
- avoidance of service locators
- avoiding dependency injection containers
- constructor injection

Many of the Composition Root principles in MEG-004 derive directly from this body of work.

Domain-Driven Design References

Domain-Driven Design

Author

Eric Evans

Referenced for:

- Bounded Contexts
- Aggregates
- Domain Services
- Repositories
- Domain independence

Although MEG-003 defines the Domain Model, Hexagonal Architecture exists largely to protect that model.

Implementing Domain-Driven Design

Author

Vaughn Vernon

Referenced primarily for:

- Application Services
- Repositories
- Aggregate persistence
- Dependency boundaries

Many practical implementation recommendations adopted by Mosaic align with this work.

Software Engineering References

Clean Architecture

Author

Robert C. Martin

Referenced primarily for:

- dependency rule
- policy vs detail
- architectural boundaries

Although Clean Architecture and Hexagonal Architecture differ in presentation, they share the central idea that dependencies point toward business rules.

The Pragmatic Programmer

Authors

Andrew Hunt

David Thomas

Referenced for:

- simplicity
- explicitness
- maintainability
- evolutionary design

Many contributor guidelines within the MEG reflect these engineering attitudes.

Refactoring

Author

Martin Fowler

Referenced for:

- architectural evolution
- continuous improvement
- incremental design

Architecture should evolve gradually.

Not through wholesale rewrites.

Go References

The Go language naturally complements Hexagonal Architecture.

Recommended references include:

Effective Go

Topics include:

- interfaces
- package design
- composition
- dependency management

https://go.dev/doc/effective_go

Go Code Review Comments

Topics include:

- interfaces
- package ownership
- naming
- architecture

https://go.dev/wiki/CodeReviewComments

Go Blog

Recommended topics:

- interfaces
- composition
- testing
- dependency management

The implementation should remain idiomatic Go while preserving architectural principles.

Reactive Runtime References

MEG-004 intentionally integrates with MEG-002.

Relevant references include:

- Event-Driven Architecture
- Reactive Systems
- OpenTelemetry
- Enterprise Integration Patterns

The Runtime remains outside the Hexagon.

It communicates with the Domain exclusively through Ports and Adapters.

Internal Mosaic Specifications

The following specifications complement MEG-004.

Engineering

- MEG-001 Go Engineering Standards
- MEG-002 Reactive Runtime
- MEG-003 Domain-Driven Design

Planned Engineering Specifications

- MEG-005 Extension Platform
- MEG-006 Runtime Architecture
- MEG-007 Storage Architecture
- MEG-008 Observability
- MEG-009 Security
- MEG-010 Performance Engineering

Mosaic Design Language

- MDL-001 Vision
- MDL-002 Principles
- MDL-003 Mental Model
- MDL-004 Interaction Model
- MDL-005 Composition Model

Mosaic Design Specifications

- MDS-001 Design Token Architecture
- MDS-002 Colour System
- MDS-003 Material System
- MDS-004 Typography System
- MDS-005 Motion System
- MDS-006 Composition Engine
- MDS-007 Tile Framework
- MDS-008 Component Library

Architectural Principles

The Hexagonal Architecture established throughout MEG-004 intentionally builds upon several enduring principles.

These include:

- The Domain owns business behaviour.
- Dependencies always point inward.
- Infrastructure remains replaceable.
- Ports belong to the Domain.
- Adapters own technology.
- Composition is explicit.
- Technology is temporary.
- Business knowledge is permanent.
- Testing is a consequence of good architecture.
- Simplicity scales better than cleverness.

These principles should remain considerably more stable than the technologies implementing them.

Keeping References Current

Software architecture continues to evolve.

Implementation techniques improve.

Frameworks come and go.

The references contained within this document SHOULD therefore be reviewed periodically to ensure:

- guidance remains relevant
- obsolete practices are removed
- better implementation advice is incorporated

The underlying architectural philosophy should remain stable even as implementation techniques evolve.

Closing Statement

MEG-004 does not attempt to invent a new architecture.

Instead, it applies the proven principles of Hexagonal Architecture to a modern, event-driven, extension-first platform.

The resulting architecture intentionally emphasises:

- explicit dependencies
- replaceable infrastructure

- protected business logic
- long-term maintainability
- technology independence

Every future engineering specification builds upon these foundations.

Because in the long life of a software platform:

> ***Technologies are temporary.***
>
> ***The business is permanent.***

Architecture exists to ensure the second survives the first.

Review Status

Status

Draft

Owner

Lead Software Architect

Previous File

`glossary.md`

Next File

End of Specification